FH | JOANNEUM
Energy and Transport Management

Master program
„Energy and Transport Management“

Kapfenberg

Master Thesis

created with the purpose of obtaining the academic degree
Master of Science in Engineering (MSc)

Driver Distraction Detection Using YOLOv11

submitted by:
Ariya Ketabi

Supervisor:
Dipl. Ing. Martijn Kiers

Kapfenberg, 3 July 2025

Declaration of authorship

I hereby declare under oath

that I have independently prepared this Master thesis and have performed all associated tasks myself, using no other sources or aids than those indicated.

that in preparing the thesis I have adhered to the guidelines of FH JOANNEUM for ensuring good scientific practice and for avoiding misconduct during the preparation of this work,

that I have properly cited all formulations and concepts taken over from printed, un-printed works as well as from the Internet in wording or in the essential content in accordance with the rules of Good Scientific Practice (guideline GSP) and have marked them by precise references,

that I have declared in the method presentation or an index all aids used (artificial intelligence assistance systems such as chatbots [e.g., ChatGPT], translation applications [e.g., DeepL], paraphrasing applications [e.g., Quill bot], image generator applications [e.g., Dall-E], or programming applications [e.g., Github Copilot], and indicated their usage at the corresponding text passages.

that this original thesis, in its current form, has not been submitted to any other aca-demic institution for the purpose of obtaining an academic degree.[1]

I have been informed that my work may be checked for plagiarism and for third-party authorship of human (ghostwriting) or technical origin (artificial intelligence assistance systems).

I am aware that a false statement may result in legal consequences such as a negative assessment of my work, the subsequent revocation of any obtained degree, and legal prosecution.

Signer ID: B4YLXDPL15...

Date, Name of the student

[1] This wording does not apply to joint study programs (e.g., double degree programs); the regulations stipulated for these apply.

Preface / Acknowledgement

First and foremost, I would like to thank my supervisor, Dipl. Ing. Martijn Kiers, for his expert guidance, constructive feedback, and continuous support throughout this program and research.

In addition, I am sincerely grateful to Mag. Silke Glatzer from the International Office for her kind assistance, which helped make my academic and cultural experience.

Special thanks also go to Ms. Claudia Rabelhofer, Program Administrator, for her constant support and helpful coordination throughout the course of my studies.

Furthermore, I would like to thank my family and friends for their patience, encouragement, and unwavering support during this academic journey.

Finally, I also acknowledge the use of AI tools to enhance the overall language quality of this thesis.

Kapfenberg, 3 July 2025

Ariya Ketabi

Abstract

Road accidents are a major concern worldwide, with millions of lives lost or affected each year. Distracted driving is one of the most significant contributors to these accidents. With the rise of smartphones, in-car entertainment systems, and other sources of distractions, more drivers are failing to pay full attention to the road. However, recent developments in technology, especially artificial intelligence, offer the potential to reduce accidents by detecting in-car distractions in real-time. Current research focuses on using the YOLOv11 model to identify these distractions under different circumstances. The model was tested on a custom data set containing various types of distractions. To assess its performance, statistical measures such as precision, recall, and F1-score were used. An analysis of training and validation losses over 50 epochs shows that the model improves over time and works well with new data. With a macro precision of 96%, the model performs strongly across all categories, highlighting its potential for use in real-world driver monitoring systems to reduce distractions and improve road safety. However, it faces challenges in distinguishing between some classes.

Keywords: Distracted driving, YOLOv11, Artificial intelligence, Driver monitoring, Road safety

Table of contents

1 Introduction 1
1.1 Problem definition 2
1.2 Research Questions 2
1.3 Objectives 2
1.4 Methodology 2
2 Theoretical background 4
2.1 Distraction: definition, causes, and types 4
2.1.1 Definition of distraction 4
2.1.2 Cause of distraction 4
2.1.3 Types of driver distraction 6
2.2 Automation levels of vehicles 7
2.3 Sensor technologies and ethical considerations 8
2.3.1 Physiological sensors 9
2.3.2 Visual sensors 10
2.3.3 External sensors 11
2.3.4 Sensor evaluation 12
2.3.5 Privacy concerns in vehicle data collection 13
2.4 Computer vision in road safety 13
2.4.1 Traffic sign recognition 13
2.4.2 Lane departure warning system 13
2.4.3 Pedestrian Detection 14
2.4.4 Object detection for collision avoidance 14
2.4.5 Automatic emergency braking system 14
2.4.6 Driver behavior monitoring 14
2.4.7 Traffic flow monitoring 15
2.4.8 License plate recognition system 15
2.5 Classical Machine Learning 16
2.5.1 Classical Machine Learning Algorithms 18
2.6 Deep Learning 20
2.6.1 Key Differences Between Deep Learning and ML 21
2.6.2 Convolutional neural networks 22
2.6.3 Image Classification Architectures 25

2.6.4 Object Detection Architectures 27
2.6.5 YOLO: You Only Live Once 28
2.6.6 Evolution of YOLO Models 33
2.6.7 Methods for improving ML techniques 38
2.6.8 Algorithms evaluation 39
2.7 Monitoring Techniques and Action Classification 40
2.8 Previous Work 43
3 Methodology 45
3.1 Literature Search 45
3.2 Dataset Description 45
3.2.1 Dataset Breakdown 45
3.2.2 Annotation Format and Object Detection Task 46
3.2.3 Image Characteristics 46
3.3 Data partitioning and model configuration 46
3.4 Installing requirements and starting training the models 46
3.5 Model Training 47
3.6 Evaluation Metrics and Statistical Analysis 47
3.6.1 Confusion Matrix 47
3.6.2 Evaluation Metrics for Model Performance 48
4 Results 51
4.1 Training Results 51
4.2 Test results 53
4.3 Experiment 56
5 Conclusion 58
References 59
List of figures 63
List of tables 64
Formula directory 65
List of abbreviations 67

1 Introduction

Road safety has always been a significant issue for societies worldwide due to the negative social and economic effects of road accidents. Each year, approximately 1.19 million people lose their lives in these incidents, with millions more suffering injuries or disabilities. The impact of these injuries extends beyond the individual involved; it also affects families, communities, and entire nations through economic aspect and increased pressure on healthcare systems. The costs associated with medical care, lost productivity, and the devastation of lives can account for about 3% of GDP, particularly in developing countries[1]. For example, in Austria, the total cost of road crashes in 2021 was approximately 11.2 billion EUR, or 2.8% of GDP [2].

Although road traffic injuries mainly impact younger people, road accidents are the leading cause of death among children and young adults aged 5 to 29. Furthermore, approximately 66% of all road traffic fatalities occur among individuals aged 18 to 59 years. This demographic is an important group, as they represent the future workforce and leaders. Losing them has significant long-term effects[1].

Many factors cause road accidents, which can be grouped into environmental, mechanical, and human errors causes. Environmental causes include bad weather such as rain, fog, or icy roads, reduce visibility and traction. It makes it harder for drivers to control their vehicles and increases the likelihood of accidents. Mechanical issues, like faulty brakes or engine malfunctions, can also cause uncontrollable situations on the road. Human errors, including misjudging distances, distraction, making poor decisions in traffic, or failing to follow road signs and signals, are another significant factor. Crash data shows that distraction contributes to about 11% of all road accidents[1].

Additionally, a study suggests that driver distraction may contribute to as much as 23% of crashes. In many countries, driver distraction is now seen as a major cause or is increasingly identified as a key cause of road accidents, just as serious as speeding, alcohol use, and driver fatigue[3], [4], [5].

Mobile phones are a major source of distraction for drivers, but they are just one of many sources that can cause distraction. Other distractions have been around for a long time, such as talking to passengers, reaching for items, adjusting the radio, smoking, eating, grooming, or even daydreaming. External distractions, such as billboards, messages on buses and taxis, or roadside signs and looking at the people on the street, can also take a driver’s attention away from the road. While not all distractions are equally dangerous or directly related to driving, each one can affect a driver’s focus in a different way and put safety at risk[5], [6].

Many accidents can be prevented with effective measures such as better urban planning, stricter enforcement of traffic laws, and public awareness campaigns and the integration of in-car systems and sensors, like cameras that detect driver fatigue or distraction. However, these actions alone may not be enough to address the issue of driver distraction. Finding effective ways to detect and reduce driver distraction is crucial for lowering the number of accidents caused by this problem. Specifically, identifying when a driver is distracted can help reduce accidents and improve safety. As technology continues to progress, innovative solutions like artificial intelligence and advanced computing techniques offer new opportunities to tackle these challenges. Specifically, due to the promising results of artificial intelligence, it has gained massive attention not only from the research community but also from the business

and government sectors. As a result, this research will investigate the use of computer vision (CV) and deep learning (DL) to detect driver distraction in real time and improve traffic safety.

1.1 Problem definition

Driver distraction is one of the leading causes of road accidents which means a threat to road safety. With the increasing prevalence of distractions, such as mobile phone usage, in-car technologies, and environmental stimuli, traditional methods of monitoring driver behavior, such as manual observation, paper-based driver logs, and post-incident reports, are often insufficient to provide real-time and accurate detection.

With the development of algorithms and advanced computational hardware, there is now an opportunity to address this issue using more sophisticated tools. In particular, the integration of artificial intelligence, especially through DL algorithms, presents a promising solution. YOLO (You Only Look Once) algorithm is one of the new techniques in CV, which has attracted attention for its real-time object detection capabilities. However, the effectiveness of the latest YOLO versions in detecting driver distractions has not yet been fully explored.

1.2 Research Questions

This study aims to answer the following research questions:

Q1: How effectively can the latest YOLO algorithm identify various driver distractions using camera images?

Q2: How accurate is the YOLO algorithm in detecting each type of distraction, and which specific distractions present the greatest challenges for detection?

Q3: How do different environmental conditions influence the performance of the latest YOLO algorithm?

1.3 Objectives

The main objective of this study is to enhance road safety using artificial intelligence by detecting and analyzing driver distractions in real time.

To support this goal, the specific objectives are:

Objective 1: To evaluate YOLO's ability to detect driver distractions from camera images, across diverse driving scenarios.

Objective 2: To assess YOLO's detection performance in terms of accuracy and limitations, using statistical analysis.

Objective 3: To identify the technical challenges and opportunities associated with implementing YOLO in practical driver monitoring systems.

1.4 Methodology

The methodology of this study includes four main steps: reviewing the literature, preparing the dataset, training the model, and evaluating the results. First, a literature review was conducted to find academic papers on the definition of driver distraction, data collection, detection methods, especially those using DL and YOLO models, and evaluation of algorithms. Trusted search engines like Google Scholar were used to search for books, journal articles and conference papers using specific keywords. Next, a dataset was used that included different types

of distractions, such as texting or talking on the phone. These images were divided into training, validation, and testing sets. A configuration file was also created to help the YOLOv11 model understand the dataset.

The model was trained using the Ultralytics version of YOLOv11 with selected settings like image size and number of training rounds. The results were checked using a confusion matrix and different evaluation metrics such as accuracy, precision, recall, and f1-Score. These metrics were used to measure how well the model could recognize each type of distraction.

2 Theoretical background

This chapter explores the foundational concepts of driver distraction, including its causes, types, and various manifestations, as well as its impact on road safety. It provides a review of the technologies and methods currently employed to detect and monitor driver distraction, such as sensors, cameras, and advanced data collection techniques. Key considerations, including sensor performance, accuracy, and privacy concerns, are also addressed to provide an overall understanding of the topic. Emphasis is placed on machine learning (ML) and DL, which form the foundation for the machine vision framework YOLO. YOLO has gained prominence for its effectiveness in real-time detection tasks. This discussion sets the stage for understanding the potential and limitations of these technologies in enhancing traffic safety.

2.1 Distraction: definition, causes, and types

In the following sections, the concept of driver distraction will be examined in detail, including its definition, causes, various types, and the potential impacts on driving performance and road safety. Understanding these elements is essential for identifying the factors that compromise driver attention and contribute to traffic incidents.

2.1.1 **Definition of distraction**

Distracted driving is defined as any activity that diverts a driver's attention from the primary task of driving, which can impair their ability to focus on the road and respond to potential hazards[5]. Driver distraction can result from two types of sources: distractions from external social roles and distractions within the driving task itself. External distractions happen when a driver’s attention shifts away from driving, often because driving does not always require full focus. Activities like social interactions or multitasking can cause attention lapses. Driving-related task distractions, such as checking a warning light or using a navigation device, also divert attention from the road, but they are sometimes necessary for safe driving. These types of distractions highlight the limitations of human attention and how competing demands can lead to lapses in focus[5].

2.1.2 **Cause of distraction**

Driver distraction can arise from many different factors. These factors may sometimes overlap, but all have been shown to reduce a driver’s ability to focus and pay attention[5].

Hands-free cell phone

The use of a cell phone while driving cause cognitive distraction, mainly by diverting attention from the driving environment. Even when using hands-free devices, engaging in a conversation on a cell phone leads to blindness, a phenomenon where drivers fail to process visual stimuli, such as traffic signals or traffic signs[7].

The dual-task nature of driving and conversing on a cell phone places substantial demands on the driver’s attention. Unlike passive auditory tasks, such as listening to the radio, cell phone conversations require active cognitive engagement, which competes for mental resources that would otherwise be allocated to monitoring the driving environment. This competition for attention results in degraded performance, including slower reactions to visual cues and a failure to detect relevant driving signals in real time, which can compromise driving safety[7].

This shift in focus is especially dangerous because drivers often do not realize how distracted they are. They may think their driving is not affected by the conversation, not noticing that their attention has shifted away from the road toward to the conversation. Studies show that using a cell phone while driving impairs judgment as much as alcohol level at the legal limit, greatly increasing the risk of accidents [7]. This highlights the serious safety risks of using cell phones while driving and more public awareness about the dangers of distracted driving.

Hand-held cell phone

Using a hand-held cell phone greatly increases the risk of crashes, near-crashes, and other dangerous situations. Tasks like dialing or texting, which require both looking at the phone and using your hands, are risky. Drivers often take their eyes off the road which leads to unsafe situations. While simply talking on a hand-held phone does not seem to raise the risk as much, the manual tasks cause distraction and reduce attention to driving [8]. Typing while driving, even briefly, is extremely dangerous. At 50 km/h, your car covers almost 14 meters every second, so just a few seconds of distraction means you're driving tens of meters without watching the road. In 2019, cell phone use was reported as a distraction in 387 fatal crashes, resulting in 422 fatalities, highlighting the significant danger of using electronic devices while driving[9].

Clothing, hair, or makeup

The activities of bringing in order of Clothing, hair, or makeup involve visual and manual distractions, as the driver must divert attention to adjusting their appearance. This type of distraction can be particularly hazardous when drivers are engaged in grooming activities that require significant focus. To be more specific, according to the 2010 NHTSA (U.S. Department of Transportation, National Highway Traffic Safety Administration) national phone survey on distracted driving, 6% of drivers reported engaging in personal grooming—such as combing hair, applying makeup, or shaving—while driving on at least some trips[10].

GPS, radio

Using in-car technology like GPS or the radio involves both visual and manual distractions. Drivers often need to take their eyes off the road to adjust settings, which can lead to missing important information from their surroundings.

While in-vehicle navigation systems are helpful for providing directions, especially in unfamiliar areas, they also pose safety risks. These distractions can take several forms. Manually entering a destination requires using a keyboard or touchscreen, pulling the driver’s focus away from the road. Visually, drivers may become distracted by looking at the screen to input information, view maps, or follow directions[11].

Even when a vehicle’s touchscreen is integrated to control essential functions like adjusting the windshield wipers. If interacting with the screen leads to a loss of focus on the road and results in an accident, the driver can be held legally responsible.

Auditory distractions

Sounds other than those necessary for driving (passengers, or external noises) can divert the driver's attention away from the road. Auditory distractions may interfere with the driver's ability to hear important sounds, like car horns or approaching vehicles, which can increase the risk of accidents.

In a study investigating the effects of distraction on driving performance, auditory (acoustic) distractors were shown to impair response times to critical events, although their impact was less pronounced than that of visual distractors. Nevertheless, auditory distractions, particularly those requiring cognitive engagement, such as verbal responses, still lead to increased braking response times and error rates[12].

Eating or drinking

Daily activities such as eating and drinking can significantly affect driving performance. Research has shown that these behaviors lead to greater deviations from the vehicle's lateral position, lower speeds, and an increased risk of crashes and near crashes. Drivers are also more likely to look away from the road while eating or drinking. Additionally, they tend to look away more often when reaching for objects or when distracted by external thoughts. Studies have further demonstrated that drivers who engage in eating and drinking behaviors are more prone to collisions, pedestrian strikes, and crossing the center line compared to those who do not engage in these activities[13].

Passenger-related distractions

Passenger-related distractions are a common risk factor that affects driver behavior and safety, leading to slower reaction times and more severe crashes. These distractions are linked to a number of road traffic incidents, though the exact impact of passenger age, especially children and teenagers, remains unclear due to differences in study designs and measures. A meta-analysis was conducted to analyze relevant studies, revealing that passenger distractions contribute to 3.55% of all crashes[14].

Reaching or turning

Reaching for items inside the vehicle or turning to attend to something in the back seat introduces manual distractions, as well as potential visual distractions if the driver has to look away from the road to retrieve or adjust something. These actions can lead to loss of vehicle control and delayed response times. Teens who reach for objects, such as food, while driving experience a nearly sevenfold increase in crash risk, attributed to the dual distractions of diverting their eyes from the road and removing their hands from the wheel[15].

Other visual/external distractions

These distractions involve anything in the external environment that competes for a driver's attention, such as billboards, accidents, or pedestrians. These visual distractions can cause the driver to lose focus on the road ahead, increasing the likelihood of missing important cues or reacting too slowly to hazards. Roadside digital billboards are a growing concern for driving safety, as they distract drivers by drawing their attention away from the road. Studies reveal that distractions causing drivers to look away for more than two seconds are responsible for up to 10% of all traffic incidents. Teen drivers are particularly affected, spending significantly more time looking at billboards compared to other age groups. The presence of digital billboards is correlated with an increase in crash rates, especially for rear-end and sideswipe accidents. These findings highlight the need for further research and stricter regulations to mitigate the risk of visual distractions posed by digital billboards[16].

2.1.3 Types of driver distraction

Driver distractions are classified into three categories based on the type of activity causing the distraction[17]. Often, these distractions overlap. For example, texting involves a visual and

manual distraction, but it also creates a cognitive distraction since the attention shifts from the road to composing or reading a message.

Manual distraction

Manual distraction refers to any action that causes the driver to remove their hands or feet from the primary controls of the vehicle, such as the steering wheel or pedals. When manual distractions occur, the driver’s ability to maintain control of the vehicle is impaired so it increases the risk of accidents[17].

Visual distraction

Visual distraction occurs when the driver loses visual contact with the road or their surroundings[17]. This reduces the driver's ability to see changes in traffic conditions, potential hazards, and the behavior of other road users, so it increases the likelihood of accidents. Maintaining continuous visual attention on the road is crucial for ensuring safety while driving[17].

Cognitive distraction

Cognitive distraction happens when a driver’s attention is mentally diverted away from the task of driving. While the driver may still be physically engaged with the vehicle, their mental focus is not on the road, which can delay reaction times and impair judgment. Cognitive distractions diminish the driver’s ability to process essential information and respond to hazards effectively, thus increasing the risk of collisions[17].

2.2 Automation levels of vehicles

Society of Automotive Engineers International defines six levels of vehicle automation, outlined in Table 1, from no automation to complete automation [18].

- Level 0: At this level, there is no automation, and the driver is fully responsible for always controlling the vehicle. Distraction detection methods are effective, as the driver must remain completely engaged in the driving task.
- Level 1: Some automation is introduced, such as control of steering or speed regulation. Traditional distraction detection methods become less effective, and new approaches, like visual monitoring, are needed to assess the driver's state. While the system assists with certain tasks, the driver still needs to maintain control and attention.
- Level 2: Partial automation is implemented, with the vehicle capable of managing both steering and speed regulation. Although some driving functions are automated, the driver must stay engaged and ready to intervene if needed. Distraction detection systems must adapt to this change, as traditional performance-based indicators, like lane control, are no longer entirely relevant.
- Level 3: At this level, the vehicle handles most driving tasks autonomously, but the driver must still be prepared to intervene when requested by the system. The challenge lies in ensuring the driver can respond promptly and take control when necessary, even though they are not required to be constantly engaged with the environment.
- Level 4: Full automation is in place for most driving tasks, and the vehicle requires no driver involvement for typical operation. Distraction detection systems are no longer necessary, as the vehicle can function independently without needing the driver's attention.

- Level 5: The vehicle is fully automated with no human involvement required in any aspect of driving. There is no need for distraction detection systems, as the vehicle operates autonomously, requiring no driver oversight.

Table 1: Levels of vehicle automation[18].

Level	Description
Level 0	No Automation: Human driver is fully responsible.
Level 1	Driver Assistance: System controls either steering or acceleration.
Level 2	Partial Automation: System controls both steering and acceleration, but driver must stay engaged.
Level 3	Conditional Automation: System handles driving in certain situations, but driver must be ready to take over.
Level 4	High Automation: System controls all tasks, even without driver response.
Level 5	Full Automation: System controls all tasks in any condition, no human involvement needed.

As automation levels progress, the need for distraction detection systems decreases. However, maintaining driver engagement and implementing effective distraction detection remains essential for Level 0 to Level 2 and Level 3 vehicles, where the risk of distraction-related accidents persists. Beginning with Level 3 vehicles, drivers are no longer required to continuously monitor the driving environment. Although the system may occasionally request driver intervention, it must allow sufficient time for the driver to re-engage with the driving task. This contrasts with lower automation levels, where drivers must be ready to take control immediately and always remain attentive. As a result, Level 3 vehicles can accommodate some degree of driver distraction. The key challenge, therefore, is ensuring the driver can respond promptly when a handover is needed. To address this, monitoring systems must detect different indicators of distraction, such as confirming the driver remains seated and ready to take over, even when not actively involved. In contrast, Level 4 and 5 vehicles are fully autonomous and capable of handling all aspects of driving without human involvement. Consequently, driver distraction detection is unnecessary at these automation levels[20]

Some car manufacturers have begun integrating driver monitoring systems that use in-vehicle cameras to track the driver’s attention. For example, DS Automobiles offers a Driver Attention Monitoring system that analyzes the driver’s face for signs of fatigue or inattention. Lexus employs a Driver Attention Monitor, using cameras positioned between the steering wheel and dashboard to track the driver’s gaze. General Motors incorporates a driver attention detection system within its Super Cruise system, which operates at Level 2 automation. This system uses an infrared front-facing camera to monitor the driver’s gaze, functioning as an advisory tool to ensure the driver stays engaged with the driving task even during partially autonomous highway driving. These technologies highlight the growing role of driver monitoring systems in maintaining safety as vehicles transition to higher levels of automation[18].

2.3 Sensor technologies and ethical considerations

The first step in detecting driver distraction is collecting input data from different sources. These sensors gather various types of information to determine if the driver is distracted. They can be grouped into three main categories: physiological, visual, and external sensors[18].

Sensors used for monitoring driver behavior can be also classified into two categories including intrusive and non-intrusive.

Intrusive sensors

Intrusive sensors are those that directly interact with the driver, such as wearable devices or sensors attached to the body. These sensors often provide more accurate and detailed data because they engage closely with the driver. However, their direct involvement can influence the driver's natural behavior, potentially create distractions and affecting driving performance.

non-intrusive sensors

On the other hand, non-intrusive sensors do not require direct interaction with the driver. They operate passively, embedded in the vehicle, allowing them to collect data without interfering with the driver's actions. This makes non-intrusive sensors more practical and effective for use in real-world driving environments, where minimizing additional distractions is crucial[18].

These sensors are often integrated into commercial vehicles to assist with automating functions like lane-keeping assistance. The ideal approach is to use sensors that can detect driver distraction without requiring extra effort from the driver or interfering with their performance. Non-intrusive sensors, especially vision-based systems, are typically preferred because they allow continuous monitoring of the driver without causing disruptions. The main role of these input sensors is to gather raw data that helps assess whether the driver is distracted. This raw data can come in different formats, including numerical, textual, or visual[18].

2.3.1 Physiological sensors

Physiological sensors measure biological functions like brain activity, heart rate, eye movements, skin conductance, and muscle activity, helping to track a driver's alertness, fatigue, emotional state, and overall well-being. Five primary physiological sensors are commonly used to monitor a driver's condition: electroencephalogram (EEG), electrocardiogram (ECG), electrooculography (EOG), electrodermal activity (EDA), and electromyography (EMG) , as presented in Table 2.

EEG measures brain activity and detects fatigue through brainwave patterns but is difficult to use in real-world driving due to its complex setup and sensitivity to noise because unwanted signals like eye blinks and muscle movements are constantly present and often indistinguishable from real brain signals without deep analysis[19]. ECG tracks heart activity, providing insights into fatigue through heart rate and variability, but is prone to motion artifacts and can be subjective. EOG monitors eye movements, such as blink frequency and eyelid closure, which can indicate fatigue, However, distraction detection through ECG is subjective and person-dependent, and it also suffers from motion-related noise. EDA measures changes in skin conductance due to sweat gland activity, reflecting emotional and cognitive states like stress and mental workload, making it useful for detecting stress during driving. EMG records muscle contractions and can detect fatigue through muscle activity, but it is also affected by other factors such as mean absolute value which refers to average strength of the EMG signal, reflecting the intensity of muscle activity, and may not be practical for everyday use. While each sensor has strengths in monitoring driver condition, they all face challenges, including sensitivity to noise, variability across individuals, and practicality for consistent use in real-world driving environments[18].

Table 2: Summary of Physiological Sensors

Sensor	Measures	Indicates	Challenges

EEG	Brain activity	Fatigue, alertness	Complex setup, noise sensitivity
ECG	Heart activity	Fatigue (heart rate)	Motion artifacts, subjective data
EOG	Eye movements (blink)	Fatigue	Individual variability, potential distractions
EDA	Skin conductance	Stress, mental workload	Sensitive to environment, practical issues
EMG	Muscle contractions	Fatigue	Affected by external factors, impractical for daily use

2.3.2 Visual sensors

Visual sensors are an essential part of driver monitoring systems, especially those used to detect distraction and inattention. These systems often start by detecting the face and then tracking the head movement of the driver. Sometimes, face detection is also used as face tracking. In other cases, a more advanced tracking method is used after the face is first detected. If the system loses face, it goes back to detection[20].

Camera placement and lighting are also very important for visual sensors to function correctly inside a vehicle. Lighting should not harm the driver’s eyes but must give enough light for good image quality. Infrared lighting is often used to help the system detect the face even in low light conditions. The location of the camera affects how well the system can see the driver's face. For example, if the camera is placed on the dashboard, the detection accuracy can be higher than if it is placed near the front mirror. When the camera captures a near-frontal face, the system works better. Moreover, placing the camera over the steering wheel helps detect head movements like pitch, which can signal distraction. However, in this position, the steering wheel may sometimes block the view, especially when the driver turns[20].

Tracking the face and estimating the head pose are also key roles of visual sensors in these systems. Head pose estimation is the process of understanding how the head is positioned in three-dimensional space, such as up and down (pitch), side to side (yaw), and tilted (roll). Many systems work well when the face is looking forward, which is common while driving. But it is during unusual head positions that dangerous events happen, like looking away for a long time. This makes it important to have a reliable tracking system even when the driver moves their head a lot. One method is to use known points on the face to handle it, like the eyes or nose, and match them to a three-dimensional face model. Some systems only need three or four face points to estimate head direction. Other systems use more complex models and track many points on the face, like 49 facial landmarks. Though, using more than one camera could increase accuracy, especially when the face is partially hidden or turned. With more than one camera, the system can choose the best image and continue tracking without failure. Multi-camera systems have shown better results and lower error rates than single-camera systems[20].

Another important function of visual sensors is the detection and tracking of facial features, especially the eyes. These features are often used to understand the driver's attention and emotional state. Accurate detection of the eyes is important because it allows the system to

measure where the driver is looking. Eye tracking is used to understand glance behavior, which helps to detect both visual and mental distraction. There are two main types of eye detection methods: those that use infrared lighting to make the pupils shine (called the bright pupil effect), and those that use shapes and patterns in the image. Some systems use light sources close to the camera lens to produce bright pupils and detect them through image processing. Other systems use classifiers or filters to find the eyes based on the shape and appearance. These systems can work even with low-cost devices and are often tested in real cars or driving simulators. However, many challenges still exist, such as changing light conditions, shadows, and when the driver wears glasses or squints in bright sunlight. To overcome these problems, some researchers use combined methods or special filters. Others use shape analysis or symmetry to detect features even when lighting is not ideal. Some systems skip direct eye detection and only process the eye area to find signs of tiredness. Overall, despite the difficulties, visual sensors remain a powerful tool to monitor drivers and improve safety by providing continuous information about attention and alertness[20].

2.3.3 External sensors

Sensors that do not collect direct data from drivers are known as external sensors. This group includes five main types:

- steering wheel sensors
- global positioning systems
- vehicle on-board diagnostic systems
- smartphones
- exterior cameras, such as those mounted outside the cabin

Steering wheel sensors capture various metrics to detect abnormal driving patterns, such as the standard deviation of the steering angle, steering wheel reversal rate, high-frequency steering, and steering entropy. These measurements reflect driving behavior and the driver’s focus on the road. The global positioning system, a satellite-based navigation system, records vehicle location and supports the analysis of driving behaviors such as speeding, acceleration, and braking[18]. Vehicle on-board diagnostic systems gather detailed data from the vehicle’s systems, including speed, braking, engine revolutions per minute, fuel consumption, steering angle, and more. These devices interface with the vehicle’s control unit through the controller area network bus to provide a reliable and continuous stream of information. Data collected through on-board diagnostic systems offers insights into driving patterns, including eco-driving behaviors, and is often more consistent than data collected through the global positioning system[18].

Smartphones, equipped with accelerometers, gyroscopes, global positioning systems, and other sensors, are effective tools for collecting driving data. Their accessibility and affordability make them a popular choice for research. Additionally, smartphone cameras can capture images and videos both inside and outside the vehicle, enabling data collection for various purposes, including distraction analysis and the creation of visual datasets[18].

Exterior cameras, such as dashcams or cameras mounted on the windshield, provide data for assessing safe driving practices. Metrics like the distance to the vehicle ahead help evaluate safe following distances, while on-road detections, such as lane recognition and traffic sign

detection, contribute to understanding driving behavior and support self-driving technologies[18].

Table 3: Summary of external sensors

Sensor Type	Data Collected	Purpose/Use
Steering Wheel Sensors	Steering angle, reversal rate, frequency	Detect abnormal driving and assess focus
Global Positioning System	Speed, acceleration, braking	Analyze driving behavior
On-Board Diagnostic Systems	Speed, braking, fuel consumption, steering	Monitor eco-driving and vehicle performance
Smartphones	Acceleration, GPS, photos/videos	Research driving, distraction analysis, visual data
Exterior Cameras	Distance, lane recognition, traffic signs	Assess safe driving practices and support self-driving

2.3.4 Sensor evaluation

Among the various sensors utilized to detect driver distraction, several key criteria are used to assess their effectiveness. These primary criteria include accuracy, reliability, responsiveness, unobtrusiveness, and practicality[18]. The effectiveness of sensors used in detecting driver distraction is evaluated based on five key criteria[21]:

- **Precision**: This determines how accurately the sensor can distinguish between distracted and non-distracted driving behavior. A precise sensor must be able to differentiate between various forms of distraction and normal driving activity.
- **Robustness**: A robust sensor consistently provides reliable data in diverse conditions, such as varying light, weather, and road surfaces. The sensor should maintain its performance regardless of external factors that might affect its readings.
- **Timeliness**: The sensor's ability to deliver real-time feedback is crucial. It should provide immediate data on driver distraction, allowing for timely interventions or warnings. This ensures that distractions are addressed as they occur, potentially preventing accidents.
- **Unobtrusiveness**: Sensors must not distract or disrupt the driver. They should integrate seamlessly into the vehicle without causing discomfort, distraction, or interference with the driver's experience. The goal is to ensure that the sensor aids in distraction detection without becoming a new source of distraction.
- **Feasibility**: This criterion assesses how easily the sensor can be incorporated into production vehicles. A feasible sensor must be cost-effective, easy to install, and compatible with existing vehicle technologies. It should also be durable enough to handle the wear and tear associated with regular driving.

Each criterion ensures that the system not only provides accurate and reliable data but also integrates seamlessly into real-world driving scenarios. By focusing on these elements, sensor technologies can effectively enhance driver safety, reduce distractions, and support safer driving behaviors while remaining practical for widespread use in production vehicles.

2.3.5 Privacy concerns in vehicle data collection

Driver distraction detection systems, especially those that rely on vision-based methods, process highly sensitive personal data such as facial features, eye movements, and head orientation[22]. These data types can reveal identifiable and private details about individuals, making privacy a critical concern in the development and deployment of such systems. Traditional approaches to model training typically require collecting and centralizing large volumes of raw data from users. In the context of driver monitoring, this would mean gathering sensitive visual information from numerous drivers and vehicles, raising significant privacy, security, and ethical issues[22].

Moreover, driver distraction data may be stored across different devices, making it difficult to integrate and utilize collectively without violating privacy standards. Attempting to centralize such data can expose it to unauthorized access, misuse, or unintentional leaks, especially when dealing with biometric or behavioral indicators that are inherently personal. Protecting user privacy in this setting requires solutions that minimize data exposure while still enabling effective monitoring. One way to achieve this is by ensuring that sensitive data remains confined to the local device where it is generated [22]. This avoids unnecessary transmission or storage of personal information and limits access to only what is strictly necessary for the system to function.

2.4 Computer vision in road safety

CV has become an essential technology for enhancing road safety. It utilizes advanced algorithms and image recognition techniques to address various traffic-related challenges. By integrating CV systems into vehicles and traffic infrastructure, significant improvements can be made in traffic management, accident prevention, and driver assistance. These systems facilitate real-time analysis of road conditions, driver behavior, and the surrounding environment, thereby contributing to the development of a safer, more efficient transportation ecosystem.

2.4.1 Traffic sign recognition

Traffic sign recognition enhances road safety through intelligent transportation systems with real-time detection and interpretation of traffic signs. With the rise in car ownership and traffic accidents, these systems assist drivers by automatically identifying and interpreting traffic signs, which provide crucial road information. The systems use CV techniques to detect signs based on their color and shape, though challenges such as changing lighting conditions, obstructions, and motion blur can complicate recognition. Traffic sign recognition typically involves detecting potential signs, tracking them across frames, and classifying their meanings, ultimately making driving safer by providing timely and accurate information to drivers[23].

2.4.2 Lane departure warning system

A lane departure warning system is an important application of machine vision designed to prevent dangerous driving situations caused by unintended lane departure. This system is particularly useful for addressing issues such as driver fatigue, distractions, or inattention, which can lead to a vehicle drifting out of its lane. Unlike autonomous driving systems, which actively control the vehicle, the Lane Departure Warning System assists the driver by detecting when a vehicle deviates from its lane and providing an alert. The system works by analyzing lane marks on the road, which are typically painted in brighter colors. Various factors, including road conditions, weather, and the degradation of lane markings, can complicate the recognition

process. A key aspect of this system involves measuring changes in the orientation of lane markings in images captured by a camera, which helps detect lane departures without being affected by camera specifications or road parameters. This approach provides a robust and efficient solution for enhancing road safety by alerting drivers to unintentional lane changes[24].

2.4.3 Pedestrian Detection

Detecting and tracking pedestrians is a critical application of CV, particularly in enhancing road safety. Pedestrian detection systems use CV to identify people in a vehicle's environment, helping prevent accidents and reduce fatalities. In the United States, nearly 5,000 out of 35,000 annual traffic fatalities involve pedestrians, emphasizing the importance of automated pedestrian detection systems. These systems are increasingly integrated into advanced vehicle safety systems, alerting drivers or even taking control to prevent collisions. By enabling machines to recognize and respond to pedestrians, CV plays a vital role in improving traffic safety and reducing pedestrian-related accidents[25].

2.4.4 Object detection for collision avoidance

CV is essential in collision detection and avoidance systems, especially in urban environments where unexpected conditions can lead to dangerous accidents. The system detects obstacles in the vehicle’s path, classifying them as vehicles, or other objects. It employs a simplified stereovision setup to instantly identify and track obstacles, adapting its methods to handle different types of objects. Additionally, it can adjust the vehicle's speed to ensure safety and prevent accidents. As vehicles continue to travel at higher speeds and cover greater distances, such systems are critical in preventing collisions, particularly when the driver is impaired or in high-risk situations. Ultimately, these systems enhance road safety, helping to avoid accidents and paving the way for the development of more advanced autonomous driving technologies[26].

2.4.5 Automatic emergency braking system

As self-driving technology advances, the need for systems that can quickly detect obstacles and make fast decisions in emergency situations has grown. Automatic braking systems use CV to identify potential hazards, such as pedestrians or other vehicles, and respond accordingly to prevent accidents. These systems are designed to recognize when a vehicle is at risk of a collision and activate the brakes automatically. By enhancing these systems, autonomous vehicles can reduce human error and help avoid accidents, particularly in complex driving environments like urban areas. The goal is to create safer driving experiences, reduce traffic fatalities, and improve overall road safety[27].

2.4.6 Driver behavior monitoring

CV plays a crucial role in driver behavior monitoring, which is essential for enhancing road safety in both autonomous and semi-autonomous vehicles. By using advanced technologies like artificial intelligence and ML, these systems can monitor and analyze a driver's physiological indicators such as facial expressions, hand placement on the wheel, and body posture. Real-time feedback and alerts can be provided to the driver, notifying them of potentially unsafe behaviors, such as drowsiness or fatigue. This proactive monitoring helps prevent accidents caused by driver inattention or unsafe driving practices. Moreover, integrating this tech-

nology with autonomous systems can improve decision-making, ensuring safer driving experiences, especially in emergency situations where human intervention may be required. By continuously assessing and improving driver behavior, these systems contribute to creating a safer and more efficient driving environment[28].

2.4.7 Traffic flow monitoring

Traffic flow monitoring is an important part of solving traffic problems at city road intersections. Intersections are places where different roads meet, and where people, vehicles, and bicycles often change direction. Because many vehicles travel through these areas every day, intersections can become crowded, which slows down movement and causes delays. These delays reduce the overall efficiency of travel in the city. By carefully observing and recording the movement of traffic, it becomes easier to understand where delays happen and how to improve traffic signal control. Many cities still use traffic lights that follow fixed schedules, which do not change based on the actual number of vehicles on the road. This can cause longer delays because the green light may stay on for too long or not long enough. Traffic flow monitoring helps gather the right information to make better decisions about when traffic lights should change. Modern transportation systems that aim to improve safety and traffic movement depend on this kind of data[29]. To address these issues, there has been increasing interest in developing automated traffic monitoring systems that use Artificial Intelligence. These systems aim to reduce the workload of human operators and improve the efficiency of traffic management. Artificial Intelligence-powered systems not only help manage traffic more effectively, but they can also detect issues like stationary vehicles that block the flow of traffic and contribute to congestion. Additionally, these systems can track the movement of individual vehicles and identify dangerous driving behaviors, such as erratic lane changing. Recent advances in DL algorithms have further improved the accuracy of these systems, enabling them to analyze traffic data in real time. This automation reduces the reliance on human operators, lowers costs, and enhances the overall effectiveness of traffic management centers. As a result, Artificial Intelligence has become an essential tool in creating smarter, more efficient traffic management systems[30].

2.4.8 License plate recognition system

Automatic License Plate Recognition is a CV technology used to automatically identify vehicles by reading their license plates. This system captures images of vehicles using cameras, often enhanced with infrared light, to ensure visibility in various lighting conditions. The primary purpose of the system is to convert the visual data from the license plate into machine-readable text, which can then be processed using Optical Character Recognition algorithms. These systems are widely used in traffic monitoring, law enforcement, toll collection, and security applications. These systems are designed to handle different license plate formats, fonts, and positioning across regions, making them adaptable to varying standards. The technology can operate with high-speed cameras, even at speeds of up to 160 kilometers per hour, though challenges such as varying lighting conditions, weather, and angles of view can affect its accuracy. Despite these challenges, these systems have become increasingly efficient and are now integral to modern transportation infrastructure and security systems[31].

2.5 Classical Machine Learning

Machine learning (ML) is the ability of computers to learn from experience rather than relying on explicit programming. This concept was introduced in 1959 and later refined, with the definition that a program is "learning" if its performance on a task improves over time based on experience, as assessed by a performance metric[32].

ML serves as the foundation of CV, enabling systems to process, interpret, and act on visual data. By leveraging algorithms and models, ML allows machines to analyze images or video feeds, identifying patterns and extracting meaningful insights. These capabilities are critical for applications like object detection, facial recognition, and activity tracking.

ML can be categorized into several types based on the amount of supervision involved during the training process, each with different methods of learning from data.

Supervised learning is the most common type of ML, where the model is trained on a dataset that includes both input data and the corresponding correct output (labels). The input data is paired with these labels, and the goal of the model is to learn the relationship between them so it can accurately predict the output for new, unseen data set. Supervised learning can be used for a variety of tasks, such as classification, where data is assigned to predefined categories (like classifying drivers as distracted or not distracted), and regression, where the model predicts continuous values, such as forecasting house prices based on certain features. In supervised learning, the training process is guided by the labeled data, which helps the model understand how to make accurate predictions[32].

Unsupervised learning differs from supervised learning in that it does not use labeled data. Instead, the model is provided with data that has no associated output labels, and its task is to discover hidden patterns, relationships, or structures in the data on its own. This type of learning is often used for clustering, where the model groups similar data points together, or for dimensionality reduction, where the model simplifies data by reducing the number of features while maintaining important information. Unsupervised learning is useful for exploring and understanding large datasets without predefined outcomes[32].

Semi-supervised learning is a hybrid approach that lies between supervised and unsupervised learning. In semi-supervised learning, the model is trained with a small amount of labeled data combined with a much larger amount of unlabeled data. The labeled data helps guide the learning process and improve the model's performance, while the unlabeled data allows the model to generalize better by recognizing patterns in the broader dataset. This approach is especially valuable when labeling data is time-consuming or expensive, but there is still a need to leverage large amounts of unlabeled data[32].

Self-supervised learning is a special type of unsupervised learning where the model generates its own labels from the data. This process allows the model to learn useful features or representations without the need for manually labeled data. Self-supervised learning is becoming increasingly popular in fields like natural language processing and computer vision, where the model can create tasks, like predicting the next word in a sentence or filling in missing parts of an image, to learn patterns and structures in the data. It allows the model to build a rich understanding of the data that can be useful for other tasks[32].

Reinforcement learning is a different paradigm in ML where an agent learns by interacting with its environment. The agent takes actions and receives feedback in the form of rewards or

penalties based on those actions. The goal of the agent is to maximize its cumulative reward over time by learning from these interactions and adjusting its behavior accordingly. Reinforcement learning is often used in situations where decisions need to be made in sequences, such as in robotics, gaming, and autonomous vehicles, where the agent must decide the best actions to take in a dynamic, changing environment [32].

These categories represent different approaches in ML, each suited to specific types of problems and datasets. They vary in the level of supervision, from fully labeled data to completely unlabeled data, and they offer different ways to discover patterns, make predictions, or optimize decisions based on the data available[32].

[35]

Gradient descent is a widely utilized optimization algorithm designed to minimize a cost function, it is commonly applied in various machine learning tasks. The core idea of gradient descent is to iteratively adjust the parameters of a model in a way that reduces the value of the cost function, such as Mean Squared Error. To better understand this concept, consider the analogy of being lost in a foggy mountain valley, where you can only sense the slope of the ground beneath your feet. The most effective strategy to reach the bottom of the valley is to move downhill in the direction of the steepest slope. Similarly, gradient descent computes the gradient of the error function in relation to the parameter vector, moving in the direction of the negative gradient. Once the gradient reaches zero, the algorithm has identified the minimum[32].

The gradient descent algorithm starts with random initialization of the model parameters. Then, it improves these parameters gradually by taking small steps aimed at decreasing the cost function. This process continues until the algorithm converges to the minimum, at which point the cost function reaches its lowest possible value. A critical element in gradient descent is the learning rate, which controls the size of the steps taken during each iteration. If the learning rate is too small, the algorithm may converge very slowly, requiring numerous iterations to reach the minimum. Conversely, if the learning rate is too large, the algorithm may overshoot the minimum, potentially jumping to a higher point than the starting position, which can lead to divergence. Therefore, selecting an appropriate learning rate is crucial for ensuring the efficiency and success of the algorithm [32].

Despite its general effectiveness, gradient descent can face difficulties depending on the characteristics of the cost function. Cost functions may exhibit irregularities, such as local minima, plateaus, or ridges, which complicate the process of finding the global minimum. If the algorithm starts near a local minimum, it may get trapped there and fail to find the global minimum. Additionally, if the algorithm begins near a plateau, it may take an unreasonably long time to traverse this flat region and converge to the minimum. Stopping the algorithm prematurely can also prevent it from reaching the global minimum. In the case of linear regression, the cost function is convex, which means that there is only one global minimum. This property guarantees that gradient descent, if executed with a suitable learning rate and sufficient time, will always approach the global minimum. This is an important feature, as it ensures that the algorithm will eventually find the optimal solution, provided the conditions are correct [32].

Another challenge that affects the performance of gradient descent is the scaling of features. When the features of a dataset have very different magnitudes, the cost function may become

distorted, resembling an elongated bowl rather than a symmetrical one. As a result, the gradient descent algorithm may initially move in a direction that is almost perpendicular to the optimal path toward the global minimum. This can cause the algorithm to take a much longer path to converge. In contrast, when the features are scaled to a similar range, the algorithm converges more efficiently, moving directly toward the minimum[32].

Training a machine learning model is essentially a process of searching for the optimal combination of parameters that minimizes the cost function. As the number of parameters increases, the parameter space becomes more complex, and the search for the optimal solution becomes more challenging. Searching in a high-dimensional space, such as a 300-dimensional space, is considerably more difficult than searching in a lower-dimensional one. However, because the cost function for linear regression is convex, the global minimum will always be at the bottom of the bowl, making gradient descent an effective optimization method in this context[32].

2.5.1 Classical Machine Learning Algorithms

Classical ML algorithms include a variety of powerful yet simple classifiers, including Support Vector Machine, Decision Trees, and Random Forests. These algorithms are widely used for analyzing numerical data, such as information from physiological sensors, though some studies, particularly those published before 2014, have also applied them to image data[32]. In the following, an overview of classical ML algorithms will be presented.

2.5.1.1 Linear Regression Algorithm

Linear Regression is a supervised learning technique that models the relationship between continuous input and output variables by fitting a straight line to the data, assuming a linear relationship. It is commonly used in various fields, such as analyzing trends. The algorithm is easy to understand and can be effective when the relationship between variables is linear. It also helps prevent overfitting through regularization techniques. However, Linear Regression has limitations. It struggles with non-linear relationships and tends to oversimplify complex, real-world problems. While useful for basic data analysis and preprocessing, it is often not suitable for tasks that involve more intricate patterns or require more sophisticated modeling techniques[33].

2.5.1.2 Multivariate Regression Analysis

Multivariate Regression extends simple linear regression by addressing situations where the dependent variable is influenced by multiple independent variables. For example, it can be used to predict house prices based on various factors such as location, size, and amenities. This method provides valuable insights into the relationships between variables and how they collectively impact the outcome. However, its complexity requires expertise and access to large datasets to ensure accurate and reliable analysis. One challenge is multicollinearity, where predictor variables are highly correlated, which can lead to overfitting and unreliable results. Although multivariate regression is a powerful tool for solving real-world problems, interpreting its outputs and managing the complexity of the model can be difficult. Common applications include property valuation, demand forecasting, and quality control in manufacturing processes[33].

2.5.1.3 Logistic Regression

Logistic Regression is a classification algorithm used to predict categorical outcomes, such as driver distraction, by estimating the probability of an event. It can handle binary, multinomial, and ordinal outcomes, making it versatile for various types of classification tasks. The algorithm is known for its simplicity, computational efficiency, and ability to handle noisy data. However, despite its popularity in solving industry-scale problems, Logistic Regression has limitations. It can only model linear decision boundaries and may overfit if the independent variables are not carefully selected. Its practical applications are diverse, including medical diagnosis, predicting process failures, and assessing credit risk[33].

2.5.1.4 Decision Tree

Decision Trees are powerful algorithms used to solve both classification and regression problems by recursively splitting data based on decision rules. They are easy to interpret, can handle both categorical and numerical data, and perform well due to efficient tree traversal algorithms. Despite these advantages, Decision Trees are prone to overfitting, especially when the tree becomes too complex, which can lead to suboptimal solutions[33].

2.5.1.5 Random Forest

To reduce overfitting in decision trees, random forests are commonly used[33]. The random forest algorithm is a widely used method for classification and regression tasks. It builds many randomized decision trees and combines their predictions by averaging, which helps improve accuracy and reduce overfitting. This method works especially well when there are more variables than observations and is suitable for both small and large data sets. Random forests use the CART-split criterion to build trees and apply a resampling method called bagging, where trees are trained on different bootstrap samples. Although the algorithm is simple to use and effective in many fields such as bioinformatics, ecology, and economics, its theoretical understanding is still developing. Researchers continue to study its mathematical properties, including how it relates to nearest neighbor methods and the consistency of its predictions, to better understand how and why it works so well in practice[34].

2.5.1.6 Support Vector Machine

Support Vector Machine is a powerful algorithm used for both classification and regression, leveraging hyperplanes to separate data points into distinct classes. It employs kernel functions to handle non-linear separations, making it effective in cases where the data is not linearly separable. This capability allows Support Vector Machine to be robust against overfitting and effective when dealing with high-dimensional data. However, it faces challenges when working with large datasets, as the computational costs can be high. Additionally, selecting the right kernel and tuning hyperparameters can be difficult, and the algorithm is sensitive to noise in the data[33].

2.5.1.7 Bayesian Learning

Bayesian Learning is a method that updates prior probabilities with new data to produce posterior probability, allowing it to make informed predictions even with incomplete datasets. This approach is particularly effective at preventing overfitting and handling contradictory or uncertain data. However, one of its challenges lies in selecting the appropriate prior probabilities, as this can significantly influence the results. Additionally, Bayesian Learning can be computationally intensive, especially with large datasets or complex models[33].

2.5.1.8 Naïve Bayes

Naïve Bayes is a probabilistic algorithm that makes predictions based on the assumption of feature independence, using probability tables to estimate outcomes. It is straightforward to implement, scales well with data, and can handle both binary and multi-class classification problems. However, its assumption of independence between features often does not hold true in real-world scenarios, which can limit its effectiveness. Additionally, Naïve Bayes struggles with continuous features and can face challenges when dealing with very large datasets. Despite these limitations, it is widely used in applications such as recommendation systems and forecasting cancer progression[33].

2.5.1.9 K-Nearest Neighbors

k-Nearest Neighbors classify data points based on their proximity to others in the training set, without making assumptions about the underlying data distribution. This flexibility makes it effective for handling multimodal classes. However, k-Nearest Neighbors can be computationally expensive, particularly with large datasets or high-dimensional data, as it requires calculating distances between points for every prediction. Despite these challenges, k-Nearest Neighbors is widely applied in various fields, including medical diagnosis, handwriting recognition, and credit scoring[33].

2.5.1.10 K-Means Clustering

K-Means is an unsupervised learning algorithm used for clustering data into groups based on feature similarity. It is computationally efficient and easy to interpret, making it a popular choice for many applications. However, K-Means faces challenges in determining the optimal number of clusters and in handling irregular cluster shapes. Additionally, its sensitivity to initial conditions and outliers can affect the quality of the results. Despite these limitations, K-Means is widely applied in various fields, including customer segmentation, document classification, and fraud detection[33].

2.6 Deep Learning

Artificial neural networks are inspired by the structure and function of biological neurons as shown in Figure 1. Biological neurons are unique cells predominantly found in animal brains. They consist of a cell body that houses the nucleus and most of the cell’s intricate components, branching structures called dendrites, and a single elongated extension known as the axon. The axon varies greatly in length, ranging from a few times the size of the cell body to tens of thousands of times longer. At its endpoint, the axon branches into multiple extensions called telodendria, each terminating in tiny structures called synaptic terminals or synapses. These synapses connect to the dendrites or cell bodies of other neurons[32].

Neurons generate brief electrical signals known as action potentials, which travel along the axon and stimulate the synaptic terminals to release chemical messengers called neurotransmitters. Although individual neurons perform relatively simple functions, they are part of an extensive network comprising billions of interconnected neurons. Each neuron typically links to thousands of others, enabling the network to perform highly complex computations[32].

Research on biological neural networks has revealed that neurons are often organized in sequential layers, particularly in the cerebral cortex, the brain's outer layer. When a neuron re-

ceives enough neurotransmitters within a short time frame, it generates its own electrical impulses. However, certain neurotransmitters inhibit this activity, adding complexity to the network's functionality[32].

Biological neurons and DL networks share several similarities in their structure and function. In biological systems, neurons consist of a cell body (soma), dendrites, and an axon, with dendrites receiving signals, the soma processing them, and the axon transmitting the output. Artificial neurons in DL mimic this structure by receiving inputs, processing them, and generating outputs, with the input signals weighted and passed through an activation function. Neurons in the brain are connected by synapses, which transmit signals and vary in strength, similar to the weights in DL that determine the influence of one neuron's output on another's input. Both biological and artificial neurons learn by adjusting these weights based on experience, with biological learning occurring through synaptic plasticity and DL relying on backpropagation and gradient descent to minimize prediction errors. The connections between biological neurons form intricate networks that allow for complex processing and adaptation, which is replicated in DL through neural networks composed of layers of artificial neurons. These networks learn hierarchical representations of data and can model complex relationships. Finally, just as biological neurons "fire" when their input exceeds a threshold, artificial neural networks use activation functions to apply non-linear transformations to the weighted sum of inputs, enabling them to process complex data in a similar way to biological neurons.

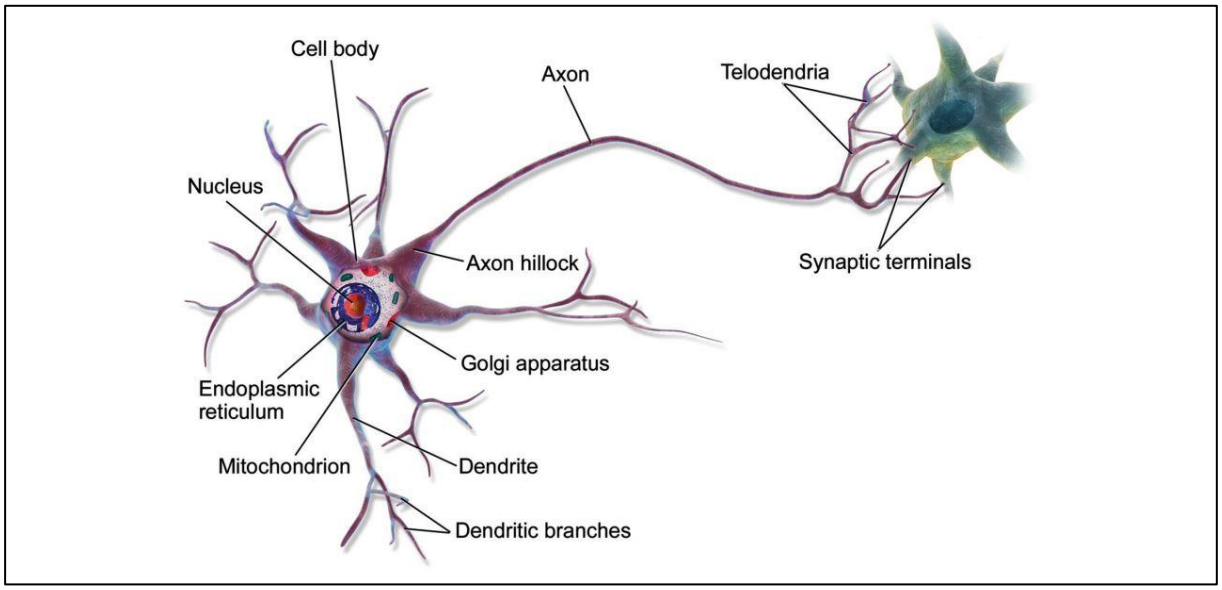


Figure 1: A biological neuron [32], [35].

2.6.1 Key Differences Between Deep Learning and ML

DL, a subset of ML, draws inspiration from the neural networks in the brain's visual cortex. Early efforts to simulate these networks on computers aimed at analyzing images that showed promising potential. However, the main challenge was the extensive computational time required to train or fine-tune the network's weights. This obstacle was largely overcome with the introduction of graphical processing units, which significantly sped up the training process by performing numerous simple calculations in parallel. This capability allowed for the exploration of larger, more complex networks with multiple hidden layers, known as DL networks [36], [37].

Unlike classical ML algorithms, such as decision trees, support vector machines, and linear regression, which rely on manually designed features and simpler models, DL models automatically extract intricate patterns from large datasets. This ability to learn complex hierarchical features directly from raw data makes DL particularly suited for tasks such as image recognition, natural language processing, and speech analysis, where traditional ML methods may fall short. The increased demand for large datasets, powerful computational resources, and extended training times further distinguishes DL from classical ML approaches. These factors have enabled DL to tackle more sophisticated and data-intensive problems, yielding superior performance in many advanced applications[36], [37].

Another name for DL is representation learning because it automatically learns useful features from raw data through multiple layers that transform the input into more abstract and informative representations. DL is based on traditional neural networks but achieves much better performance than earlier models. Furthermore, DL combines transformations with graph technologies to develop multi-layer learning models[38].

The effectiveness of an ML algorithm is typically dependent on the quality of data representation. In recent years, DL has become one of the leading research areas in ML, largely due to its significant success. Research has demonstrated that a well-constructed data representation can greatly enhance performance, in contrast to a poor representation. As a result, feature engineering has been a central focus in ML for many years, guiding numerous studies. This process involves creating features from raw data and is highly specific to the field, often requiring considerable human effort. In contrast, feature extraction in DL algorithms is automated. These algorithms use a multi-layer data representation structure, where the initial layers extract low-level features, and the final layers capture high-level features. This architecture, inspired by artificial intelligence, mimics the process in the brain's sensory regions, where data is automatically represented from various scenes. Specifically, this process leads to classified objects as output, with the scene information acting as input, mirroring how the human brain operates. This characteristic highlight one of the key advantages of DL[38].

DL is a highly effective approach in artificial intelligence, known for its versatility, robustness, and scalability[38]. It is often referred to as a "universal learning approach" because it performs well across a wide range of applications and domains. Unlike traditional methods that require carefully designed features, DL automatically learns optimized features based on the task, making it robust to changes in input data.

It also demonstrates strong generalization abilities, enabling the same techniques to be applied across various data types and applications, a concept known as transfer learning, which is particularly valuable when data is limited. DL is also highly scalable, capable of handling growing demand efficiently without losing performance. It can process larger datasets, handle more complex tasks, and operate on expansive computational systems. For example, Microsoft’s ResNet, with over 1200 layers, performs effectively on supercomputers[39]. Similarly, organizations like the Lawrence Livermore National Laboratory use DL frameworks across thousands of nodes, demonstrating its ability to manage massive, complex tasks seamlessly[40]

2.6.2 **Convolutional neural networks**

Convolutional neural network (CNN) is one of the most widely used algorithms in DL, known for their ability to automatically identify relevant features from input data without the need for

human supervision. This feature makes them more powerful than earlier models, like artificial neural networks, that required manual feature extraction. CNNs have been extensively applied in various fields, such as computer vision, speech processing, and face recognition. Their ability to process complex data like images and audio has greatly impacted these industries, allowing machines to perform tasks that were once considered to be human-specific[38].

The architecture of CNNs is inspired by the way neurons function in the human brain, particularly in the visual cortex of a cat. In the visual cortex, a sequence of cells processes visual information, each detecting different parts of a scene. This structure is mimicked in CNNs, where local connections and shared weights are used to identify patterns in the input data. Unlike traditional fully connected networks, CNNs focus on small regions of input data, such as portions of an image, which allows them to efficiently capture local correlations. This approach reduces the number of parameters needed, making the training process faster[38].

A typical CNN consists of several convolution layers followed by sub-sampling or pooling layers, with fully connected layers at the end. Each convolution layer applies a filter to the input data, extracting features like edges or textures. These filters share weights, which helps to reduce the number of parameters and improve computational efficiency. The input data is organized in three dimensions: height, width, and depth. In the case of a color image, the depth represents the three-color channels, and the convolution filters process small areas of the image, creating feature maps that represent the patterns detected[38].

After the convolution layers, the feature maps are down sampled using pooling layers, which reduce the size of the data and the number of parameters such as manual filters or massive fully connected layers. This step also helps to prevent overfitting. The final fully connected layers process the mid- and low-level features, combining them into higher-level representations. These layers generate classification scores, which represent the probability of an image belonging to a certain class. Techniques such as support vector machines, small datasets, or softmax are often used in the final layer to produce these probabilities. This process enables CNNs to classify complex data, such as images, with high accuracy[38]. To be more specific, softmax makes it easy to interpret the output as the model’s confidence that the input belongs to each class.

The use of CNNs offers several significant advantages over traditional neural networks, particularly in the field of computer vision. One of the main benefits is the weight sharing feature, which reduces the number of trainable parameters in the network. This reduction not only speeds up the training process but also helps improve the generalization ability of the model, making it less prone to overfitting. By sharing weights across different regions of the data, the network can learn more efficiently and better capture spatial hierarchies in the data[38].

Another key advantage of CNNs is their ability to concurrently learn both feature extraction layers and the classification layer. This integrated approach ensures that the output of the model is highly organized and strongly dependent on the features extracted from the input data. By learning these layers together, the network is able to create a more effective representation of the data, which enhances its accuracy and performance, particularly in tasks like image recognition where distinguishing important features is critical.

In addition, CNNs are particularly well-suited for large-scale network implementation. Compared to traditional neural networks, CNNs can handle vast amounts of data and complex

architectures more easily. The hierarchical structure of CNNs, with its layers of convolution, pooling, and fully connected layers, allows for efficient processing of high-dimensional data, such as images, while maintaining a manageable number of parameters. This makes CNNs a more scalable solution for tasks that require processing large datasets, making them ideal for applications in industries like computer vision, speech processing, and medical imaging[38].

2.6.2.1 CNNs layers

A Convolutional Neural is composed of multiple layers, each serving a distinct purpose in transforming input data into meaningful outputs like classifications or detections. These layers include convolutional layers, pooling layers, activation function layers, and fully connected layers, all of which work together to extract features and make predictions. This explanation details the essential components of a CNN and their roles in the architecture [38].

Convolutional Layer: The convolutional layer is the most fundamental component of CNN. It uses small grids of weights, called kernels, to scan over the input image and extract features. Initially, these weights are random, but they are adjusted during training to identify key patterns in the image. The convolution operation involves sliding the kernel across the image, computing dot products between overlapping areas of the image and the kernel. The result is a feature map representing extracted characteristics. Adjustments such as stride, which defines the step size of the kernel, and padding, which adds borders to the image, influence the size of the output. Sparse connections and shared weights make this layer efficient in terms of memory and computation[38].

Pooling Layer: The pooling layer reduces the size of feature maps generated by the convolutional layer while retaining critical information. This process helps lower the computational load and prevents overfitting. Common pooling methods include maximum pooling, which selects the highest value in a region, and average pooling, which computes the mean value. The pooling operation slides a small grid, similar to the convolutional kernel, across the feature map. While pooling reduces data size and complexity, it may lose some spatial information, as it focuses on the presence of features rather than their exact locations[38].

Activation Function Layer: Activation functions introduce non-linearity to the network, enabling it to learn complex patterns in the data. These functions process the weighted sum of inputs from the previous layer to determine whether a neuron should activate, producing a non-zero output in response to its input. Examples include the sigmoid function, which outputs values between 0 and 1, and the hyperbolic tangent function, which outputs values between -1 and 1. The Rectified Linear Unit, or ReLU, is particularly popular as it outputs only positive values, making computations faster. Variants like Leaky ReLU address issues such as neuron inactivity by allowing small outputs for negative inputs. Activation functions must be differentiable to enable error correction during training [38].

Fully Connected Layer: At the end of a CNN, the fully connected layer combines all the extracted features to make a final prediction or classification. Each neuron in this layer connects to all neurons in the preceding layer, ensuring every feature contributes to the decision. Input from earlier layers is flattened into a single vector before being processed in this layer. This layer functions as a classifier, assigning probabilities or labels to the data based on the learned features [38].

Loss Functions: Loss functions measure the discrepancy between the network's predictions and the actual outputs. This error is used to guide the optimization process, allowing the network to adjust its weights to improve accuracy. The choice of loss function depends on the task, such as categorical cross-entropy for classification or mean squared error for regression. By minimizing this error through training, the network refines its parameters and achieves better performance, making loss functions critical to the learning process in a CNN[38].

Table 4: Key components of neural network architecture

Component	Description	Key Features
Convolutional Layer	Extracts features using kernels	Sparse connections, weight sharing, stride, and padding
Pooling Layer	Reduces feature map size	Max/average pooling, improves efficiency, prevents overfitting
Activation Function	Adds non-linearity for complex pattern learning	Sigmoid, Tanh, ReLU, and variants
Fully Connected Layer	Combines features for final prediction	All features influence output, acts as classifier
Loss Functions	Measures error to guide learning	Cross-entropy (classification), MSE (regression)

2.6.3 Image Classification Architectures

Over time, a diverse range of CNN architectures has been meticulously designed and refined to address various challenges in CV tasks. These architectures aim to improve accuracy by enabling better feature extraction and representation, often through deeper networks, innovative layer designs, or enhanced training techniques. Efficiency has been a critical focus, with models optimized to reduce computational requirements and memory usage, making them more accessible for deployment on devices with limited resources. Scalability, ability to grow with more data and complexity while staying efficient and effective, has also been a key consideration, leading to the development of adaptable architectures capable of handling increasingly large and complex datasets while maintaining high performance.

LeNet, introduced by Yann LeCun and his colleagues in 1998[41], is one of the earliest successful implementations of CNNs, representing an important step in the development of DL. Initially designed for recognizing handwritten digits, LeNet was applied to the Modified National Institute of Standards and Technology dataset, which contains images of digits from 0 to 9. The architecture of LeNet, while simple by today's standards, was groundbreaking at the time. It featured convolutional layers for feature extraction, pooling layers for dimensionality reduction, and fully connected layers for classification. The convolutional layers allowed the network to automatically learn spatial hierarchies of features from raw pixel data, marking a shift from previous methods that relied on manually designed features. LeNet showed the power of hierarchical feature learning and backpropagation in neural networks, proving that they could learn complex visual patterns without manual feature engineering, laying the foundation for modern DL[41].

[46]AlexNet, introduced by Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton in 2012, was a transformative architecture that played a key role in the resurgence of DL in computer vision. The network was designed to compete in the ImageNet Large Scale Visual Recognition Challenge, a prestigious competition focused on classifying millions of images into a thousand different categories. [42].

The architecture of AlexNet consisted of eight layers: five convolutional layers followed by three fully connected layers. One of the key innovations in AlexNet was the use of Rectified Linear Units as activation functions. This choice helped speed up training by addressing the vanishing gradient problem commonly encountered with sigmoid and hyperbolic tangent activation functions. Additionally, AlexNet employed techniques such as dropout for regularization to prevent overfitting and overlapping maximum pooling, which improved feature extraction. The model was trained on Graphics Processing Units, significantly accelerating the computation and making it feasible to train deep models on large datasets[42].

A major impact of AlexNet was its ability to learn directly from raw pixel data, eliminating the need for manually crafted features, Scale-Invariant Feature Transform and HOG Histogram of Oriented Gradients, traditionally required in image processing. This capability laid the groundwork for the widespread use of DL across various fields. AlexNet not only demonstrated the effectiveness of CNNs but also marked the beginning of the "DL revolution," leading to the widespread adoption of neural networks in CV and other domains.

VGGNet, developed by the Visual Geometry Group in 2014 at the University of Oxford, introduced a new approach to building deeper CNNs, showing that increasing the depth of a network could improve the performance of image recognition models. Karen Simonyan and Andrew Zisserman investigated the effect of increasing the number of layers in the network while maintaining a simple and uniform architecture. The core idea behind VGGNet was to use small 3x3 convolutional filters stacked in layers. The network employed a very deep architecture with up to 19 layers, enabling it to learn more complex and abstract features from the data[43].

Unlike previous models, such as AlexNet, which used a mix of filter sizes, VGGNet used only 3x3 filters throughout the entire architecture. This simple yet effective design demonstrated that using smaller filters with more layers could result in better performance. Another notable feature of VGGNet was its consistent use of max-pooling layers after every few convolutional layers, which helped reduce the spatial dimensions of the data while preserving the most important features. VGGNet performed exceptionally well in the ImageNet competition, securing high ranks with its deep network[43].

However, the simplicity and depth of VGGNet came with a significant downside: its computational and memory requirements were extremely high. The model contained many parameters, which made it slow to train and difficult to deploy on devices with limited resources. Despite this, VGGNet has been widely used as a backbone architecture for many other tasks in DL, such as feature extraction and transfer learning, due to its strong performance and simplicity.

ResNet, introduced by Kaiming He and his colleagues in 2015 [39], brought a breakthrough in how deep neural networks can be built and trained. Before ResNet, models like VGGNet showed that increasing the depth of a network could improve its performance. However, as networks became deeper, they faced significant problems. One of the main issues was that adding more layers made training harder, leading to problems like vanishing gradients, where the gradients used in training became too small to update the model effectively. Another problem was degradation, where adding more layers actually hurt the model's performance[39].

ResNetsolved these problems by introducing a new concept called "residual connections" or "skip connections." These connections allow the network to skip one or more layers, which means it learns residual functions instead of trying to directly map inputs to outputs. This made

it easier for the network to train, as the gradients could flow more easily during backpropagation, even in very deep networks. This was a breakthrough because it showed that it was possible to train much deeper networks successfully [39].

In the 2015 ImageNet competition, ResNet achieved state-of-the-art performance, with the ResNet-152 model (152 layers deep) winning the competition. The ability to train such a deep model demonstrated that increasing the depth of a network could improve its accuracy without the performance problems that had been seen in earlier models. The idea of residual connections has since been widely adopted in other models and has become a fundamental part of modern DL architecture [39].

2.6.4 Object Detection Architectures

In 2014, Ross B. Girshick and his team introduced Region-based CNNs, which significantly advanced the field of object detection by combining CNNs with region proposal methods. Prior to this, object detection relied heavily on manually designed features and traditional ML models, such as Support Vector Machines, for classifying objects. The Region-based CNNs approach revolutionized this process by using CNNs to automatically learn features directly from raw image pixels, eliminating the need for manual feature extraction. This was a breakthrough in the field because it allowed for more accurate and efficient detection. The method involves three main steps: first, generating a set of candidate object regions from an image using a selective search algorithm, which groups pixels into meaningful regions; second, applying a pre-trained CNN, such as AlexNet, to extract feature maps for each region; and third, passing these features through Support Vector Machines to classify the objects in each region, while also refining the predicted locations of the objects using bounding box regression[44].

While Region-based CNNs showed considerable improvements in object detection compared to previous methods and achieved state-of-the-art performance on benchmark datasets, it also faced some notable challenges. One of the main issues was the slow processing speed, as each candidate region needed to be processed independently by the CNN, making the method computationally expensive. This resulted in long processing times, especially for images with many regions. Additionally, the storage requirements were high because storing the feature maps for each region took up a large amount of memory. Despite these challenges, Region-based CNNs marked a major step forward in object detection and laid the foundation for future innovations in the field, including faster and more efficient models such as Fast Region-based CNNs and Faster Region-based CNNs[44].

Fast Region-based CNNs introduced by Ross B. Girshick in 2015, builds upon the original Region-based CNN but address the computational inefficiencies of the latter. Unlike Region-based CNNs, which require separate feature extraction for each proposed region, Fast Region-based CNNs extract features from the entire image once using a CNN and then use Region of Interest pooling to classify each region. This approach reduces the amount of redundant computations and speeds up the detection process. Additionally, Fast Region-based CNNs jointly train the CNN and the classifier using a multi-task loss function, optimizing both the detection accuracy and the bounding box regressions for improved localization[45].

Fast Region-based CNNs significantly improve processing speed compared to their predecessor by eliminating the need for storing and processing large numbers of individual feature maps for each region proposal. This makes it feasible for real-time object detection applications.

Another key innovation is the use of Region of Interest pooling, which ensures that the feature map for each region is of fixed size, allowing it to be fed directly into fully connected layers for classification. Fast Region-based CNNs also benefit from end-to-end training, where all layers, from feature extraction to classification and bounding box regression, are trained simultaneously. This unified approach not only accelerates the training and testing phases but also improves performance, as the entire model is optimized for the detection task. The method set the stage for further advancements in object detection, leading to even faster and more accurate models like Faster Region-based CNNs[45].

Faster Region-based CNN, introduced by Ross B. Girshick in 2015, improves object detection by making the process faster and more efficient. Unlike previous methods, such as Fast Region-based CNN, which relied on external algorithms like selective search to generate region proposals, Faster Region-based CNN uses a Region Proposal Network to generate these proposals directly from the feature map produced by the CNN. This makes the entire process end-to-end trainable and much faster, as all steps—feature extraction, proposal generation, classification, and bounding box refinement—are handled by a single network[46].

The Faster Region-based CNN has three main parts: the CNN, which extracts features from the image; the Region Proposal Network, which creates region proposals; and the Region of Interest pooling layer, which processes each proposal and prepares it for classification and bounding box prediction. The key advantage of Faster Region-based CNN is that it trains all these components together, which reduces the computational cost and speeds up the overall process. As a result, Faster Region-based CNN is much faster than previous methods and achieves excellent performance in object detection tasks[46].

2.6.5 **YOLO: You Only Live Once**

The human visual system is capable of instantly recognizing objects, understanding their locations, and comprehending their interactions within a scene. This remarkable efficiency enables the human being to perform complex tasks, such as driving, with minimal conscious thought. Inspired by this, researchers have sought to develop fast and accurate algorithms for object detection that allow computers to perform similar tasks, such as autonomous driving and assistive technologies[47].

Object detection traditionally relied on multi-stage pipelines that separated object localization and classification. These approaches, while effective, were often computationally expensive and lacked the speed necessary for real-time applications. YOLO, proposed by Redmon et al. (2015) [47], revolutionized object detection by reframing it as a single regression task. This unified architecture enables YOLO to predict bounding boxes and class probabilities directly from image pixels in one step, significantly improving speed and accuracy[47].

2.6.5.1 Bounding Boxes and Object Detection Basics

A bounding box is a rectangle that surrounds an object in an image, showing its location, category (e.g., dog or bicycle), and a confidence score indicating the likelihood that the object is correctly detected. Bounding boxes can be defined by the coordinates of their top-left and bottom-right corners or by the coordinates of their center along with their width and height. YOLO adopts the latter format, representing bounding boxes as [x, y, w, h], where (x, y) is the center and w and h are the width and height, normalized to the image size [48].

For example, in Figure 2, a bounding box around a dog shows its position with a confidence score of 0.78, meaning YOLO is 78% sure that there is a dog and that the box correctly covers its position YOLO generates a small, fixed number of bounding boxes per image, making it an efficient system capable of real-time detection by predicting these bounding boxes and class probabilities in one pass [48].

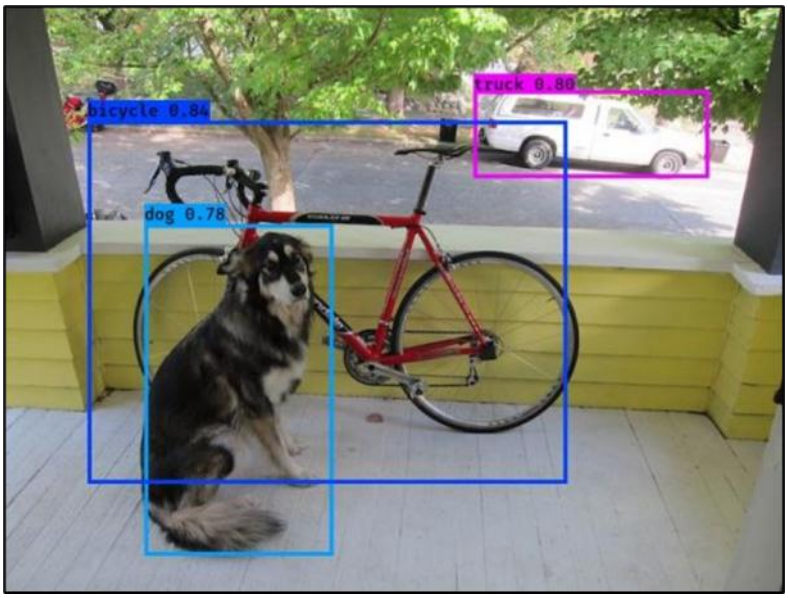


Figure 2: Object Detection in Action: A YOLOv2 Example[48]

2.6.5.2 Architecture of YOLO 1

The YOLO architecture consists of three main components: the backbone, neck, and head, which can be seen in Figure 3 [48].

Backbone: The Backbone is a deep CNN pre-trained on a classification dataset (e.g., ImageNet) for feature extraction and improves their quality by combining information from different layers.

Neck: The Neck aggregates feature maps from the backbone to enhance semantic representation, meaning it makes the features clearer and more meaningful so the model can better understand and detect objects of varying shapes and sizes.

Head: The head generates predictions, including bounding box coordinates, confidence scores, and class probabilities. removes overlapping bounding boxes that predict the same object, keeping only the one with the highest confidence score to avoid duplicates.

YOLO v1 uses 24 convolutional layers followed by two fully connected layers. The backbone processes an input image of size 448 pixels wide, 448 pixels tall, and has 3 color channels (red, green, and blue)., outputting a feature map of size 7 × 7 × 1024 as shown in Figure 4. This feature map is flattened and passed through the fully connected layers to produce a final prediction tensor of size 7 × 7 × 30 (predictions per cell) (Figure 5). This tensor includes two bounding box predictions per grid cell and probabilities for 20 object classes [48].

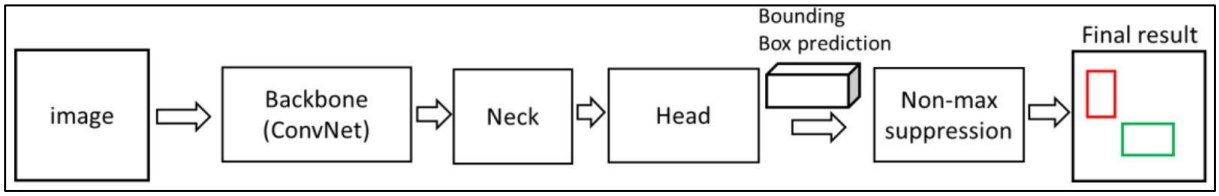


Figure 3: the generic architecture of YOLO[48]

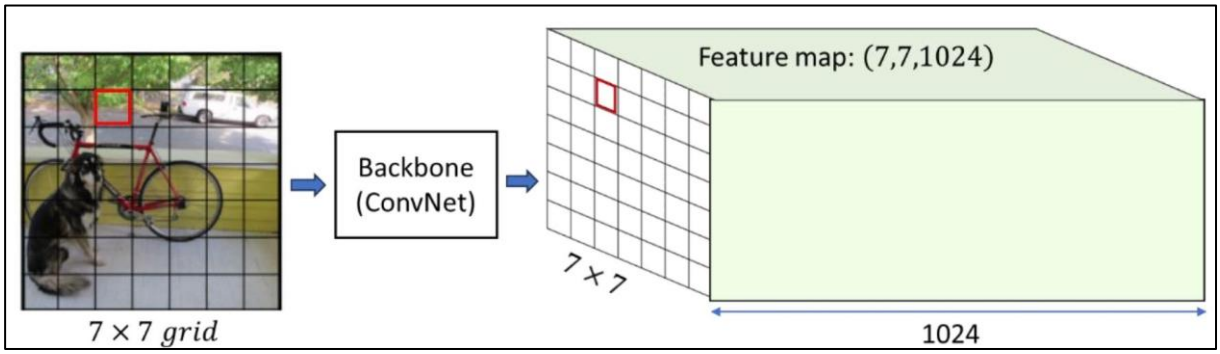


Figure 4: The backbone extracts one feature vector for each 7 × 7 grid cell[48]**.**

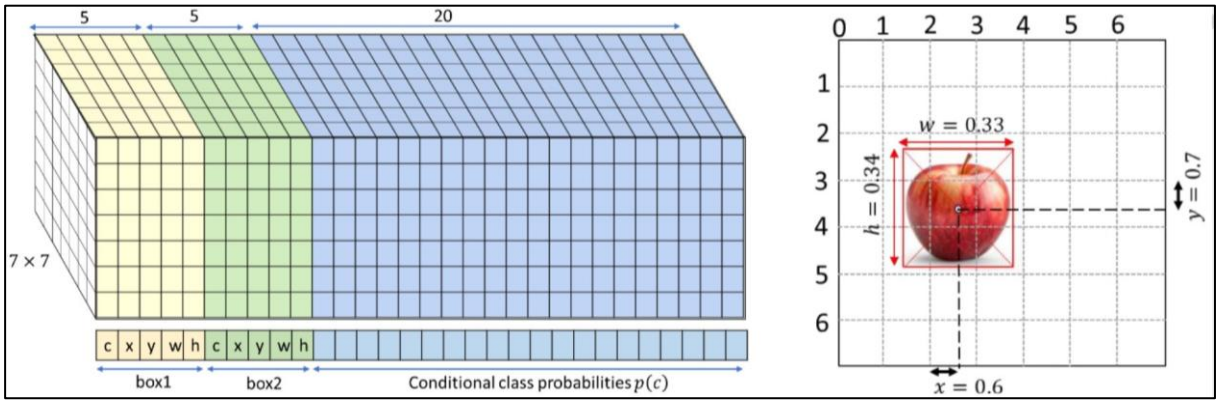


Figure 5: YOLO v1 Prediction Tensor. (Left) Tensor Shape (Right) Example of x, y, w, h

2.6.5.3 Predictions and Inference

YOLO divides the input image into a grid of size 7 × 7. Each grid cell predicts two bounding boxes, described by:

- Confidence score: Represents the likelihood of an object within the box and its overlap with the ground truth (Intersection Over Union, IOU).
- Bounding box coordinates: Center (x, y) and dimensions (w, h) normalized to the image size.
- Class probabilities: Conditional probabilities for each object class.
- During testing, YOLO multiplies the confidence score with the class probabilities to calculate class-specific scores for each box. Redundant boxes are removed using NMS, ensuring that only the best bounding box for each detected object is retained -, which is called Intersection over Union (Figure 6).

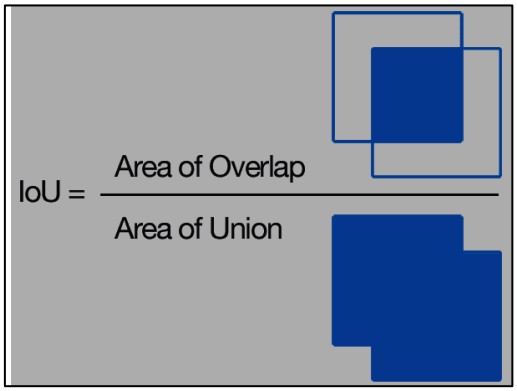


Figure 6: Intersection over Union (IOU)

2.6.5.4 Training and loss function

Yolo uses 24 convolutional layers followed by two fully connected layers, opting for simpler 1×1 reduction layers and 3×3 convolutions instead of GoogLeNet's complex inception modules. he 1×1 layers help reduce the number of feature channels, making the model faster and lighter without losing important information. The 3×3 convolution layers capture small details like edges and shapes, which are useful for understanding the content of the image. This design balances computational efficiency with performance. A faster variant, Fast YOLO, reduces the convolutional layers to 9, further enhancing speed while maintaining accuracy[49]YOLO v1 was pretrained on ImageNet for classification, using 20 convolutional layers, an average pooling layer, and a fully connected layer. This setup achieved a top-5 accuracy of 88% with 224×224 pixels input images. For object detection, the model was adapted by replacing the final layers with four convolutional and two fully connected layers, and increasing the input resolution to 448×448 pixels to improve small-object detection. The model was then trained on PASCAL VOC datasets with a custom loss function.

A) Localization loss

The localization loss evaluates the discrepancies between the predicted and actual boundary box positions and dimensions. It penalizes the bounding box localization error only if the box is considered "responsible" for the ground truth box, meaning it has the highest IOU with the ground truth box within the grid cell [48].

$$\lambda_{coord}\sum_{i=1}^{S^2}\sum_{j=1}^{B}\theta_{ij}^{obj}[(x_i-\hat{x}_i)^2+(y_i-\hat{y}_i)^2] + \lambda_{coord}\sum_{i=1}^{S^2}\sum_{j=1}^{B}\theta_{ij}^{obj}\left[\left(\sqrt{w_i}-\sqrt{w}\right)^2+\left(\sqrt{h_i}-\sqrt{\hat{h}_i}\right)^2\right] \quad (1)$$

where $\theta_{ij}^{obj}=1$ if the $j-th$ bounding box in cell i is responsible for detecting the object, and 0 otherwise. In other words, the localization error is only added to the loss for the boxes that are deemed responsible. A variable with a hat indicates a prediction (e.g., $\hat{x}_i$), while a variable without a hat represents the ground truth (e.g., x_i).

The parameter λ_{coord} (coordinate predictions, default value set to 5) increases the weight of the loss associated with the bounding box coordinates. The parameter Image (default value set to 5) increases the weight of the loss associated with the bounding box coordinates, which

means the model pays more attention to how correct the box around each object is. For example, if the model is trying to detect a dog in a picture, it not only needs to say “this is a dog,” but also draw a box that fits the dog well. If the box is too small or not in the right place, the model gets a penalty (called loss). This parameter makes that penalty stronger, so the model learns to make better boxes. Sum-squared error treats errors in both large and small boxes equally. However, the ideal loss should account for the fact that the same deviation in large boxes is less critical than in small boxes. This is because a small shift in a large box may still cover most of the object, while the same shift in a small box could completely miss important parts of the object.

B) Confidence loss

The confidence loss is applied to the boxes that are responsible for detecting an object. It evaluates the accuracy of the predicted confidence score for those boxes [48].

$$\sum_{i=1}^{S^2}\sum_{j=1}^{B}\theta_{ij}^{obj}(c_i-\hat{c}_i)^2 \tag{2}$$

where $\hat{c}_i$ is the box confidence score for the $j-th$ box in cell i. The ground truth confidence score c_i for a box containing an object is defined as the IOU between the predicted and ground truth boxes, as stated in the original paper. However, in many implementations, the ground truth confidence score is simply set to 1.

For the boxes that are not responsible for detecting an object, the confidence loss is

$$\lambda_{noord}\sum_{i=1}^{S^2}\sum_{j=1}^{B}\theta_{ij}^{nobj}(c_i-\hat{c}_i)^2 \tag{3}$$

where θ_{ij}^{nobj} is the complement of θ_{ij}^{obj}, meaning that the $j-th$ box in grid cell i is not responsible for detection.

$\hat{c}_i$ is the box confidence score for the $j-th$ box in cell i. The ground truth confidence score c_i for a box without an object is defined as 0.

λ_{noord} (default value of 0.5) reduces the confidence loss for boxes that do not contain objects. Since most boxes are background and do not contain objects, the model is trained to detect background more often than objects. To address this imbalance, λ_{noord} is used to limit the loss contributed by background detections.

C) Classification loss

If an object appears in cell *i*, the sum-squared error of the predicted class conditional probabilities in cell *i* is counted for the loss function [48],

$$\sum_{i=1}^{S^2}\theta_i^{obj}\sum_{c\epsilon classes}\left(p_i(c)-\hat{p}_i(c)\right)^2 \tag{4}$$

where:

$\theta_i^{obj}=1$ if an object appears in cell i. This implies that conditional $p_i(c)$.

$\hat{p}_i(c)$ denotes the predicted conditional class probability for class c in cell i. Given that the ground truth object belongs to a class, the ground truth probability is 1.0 for that class, and 0.0 for all others.

D) Total loss

The total loss function is the sum of the localization loss, confidence loss, and classification loss [48].

$$Total\ Loss\ =\ Localization\ Loss + Confidence\ Loss + Classification\ Loss \quad (5)$$

Note that the loss function penalizes classification errors only when an object is present in the corresponding grid cell. It also penalizes the bounding box coordinate errors only if the box is responsible for the ground truth box. Confidence score errors are penalized for both object-present and non-object-present boxes, but with different weights, where non-object-present boxes receive less weight[48].

2.6.5.5 Inference and non-maximal suppression

During inference (when the trained YOLO model is used to make predictions on a new image), YOLO predicts the location of objects by drawing bounding boxes, giving a confidence score, and estimating the likelihood of each class for each grid cell in the image. Each grid cell predicts two bounding boxes, so a total of 98 bounding boxes are generated for the whole image. To remove duplicate or overlapping detections, Non-Maximal Suppression is used. NMS keeps the bounding box with the highest confidence score and removes others that overlap too much with it. For example, if multiple boxes are predicted for the same car, Non-Maximal Suppression will keep the one with the highest score and remove the others. This helps to keep only the most accurate box, improving the results by reducing false positives[48].

2.6.6 Evolution of YOLO Models

YOLOv1 and YOLOv2 were the pioneering models in the YOLO series, both aiming to streamline real-time object detection by using a single convolutional network for both classification and localization.

The first version of YOLO has several limitations that affect its accuracy and performance. One significant issue is the strong spatial constraints on bounding box predictions. Each grid cell can only predict two bounding boxes and is limited to predicting a single class, which restricts the model's ability to detect multiple nearby objects. This is particularly problematic for detecting small objects that appear in groups, such as flocks of birds. Additionally, YOLO's architecture struggles with generalizing objects that have unusual aspect ratios or configurations because it learns bounding box predictions from data and is not as flexible in handling new or uncommon shapes. The down sampling layers used in the network result in relatively coarse features for bounding box prediction, which reduces its accuracy when detecting smaller objects. Finally, while YOLO's loss function is designed to approximate detection performance, it treats errors in large and small bounding boxes equally. This is problematic because a small error in a small box has a much greater effect on IOU than the same error in a large box, making localization errors a significant source of inaccuracy in the first version of YOLO[47].

YOLOv2 improved on this with better accuracy and faster performance. However, both models struggled with small object detection and precise localization. Neither of these versions is supported by Ultralytics, which shifted focus to later, more optimized versions.

YOLOv3 built upon its predecessors by using the Darknet-53 framework, which provided a more powerful feature extractor with 53 convolutional layers. It introduced multi-scale feature maps, improving detection across large, medium, and small objects. Despite these improvements, YOLOv3 still faced challenges with detecting medium and large objects accurately. To address these shortcomings, Ultralytics released YOLOv3-u, which incorporated an anchor-free detection method, improving accuracy and speed, especially for medium and large-sized objects [49]. An anchor-free detection method eliminates the need for predefined anchor boxes by directly predicting object centers and sizes, simplifying the detection process and reducing computational complexity.

YOLOv5, developed by Glenn Jocher, transitioned from the Darknet framework to PyTorch, integrating the CSPDarknet backbone for more efficient feature extraction and reduced computational costs. YOLOv5 introduced several augmentation techniques like Mosaic, MixUp, and HSV augmentation to improve performance on objects of varying sizes. These augmentation techniques artificially expand the training dataset by combining images, adjusting colors, and altering image composition, helping the model generalize better and become more robust to different visual variations. Ultralytics further optimized this model with YOLOv5-u, enhancing it with an anchor-free detection system, which improved overall performance, particularly on complex and multi-scale objects.

YOLOv6 emphasized real-time inference optimizations, focusing on improving the speed of object detection tasks. However, it lacks pretrained models (*.pt) in the Ultralytics library, which limits its use in certain benchmarking studies. This absence of pretrained models means YOLOv6 cannot be directly compared with other YOLO versions in performance evaluations that rely on Ultralytics-supported models[49].

YOLOv7 continued the trend of improving object detection speed and efficiency, similar to YOLOv6, but like YOLOv6, it is not supported by Ultralytics, thus excluding it from many benchmark comparisons. The lack of support for YOLOv7 in the Ultralytics ecosystem means it is often not included in studies that focus on models with consistent optimization and benchmarking[49].

YOLOv8 marked a significant evolution in the YOLO series by introducing not only object detection but also additional tasks such as semantic segmentation and image classification. This multi task capability allows YOLOv8 to not only locate and identify objects but also understand the precise shape and category of each object within an image, making it a more versatile and powerful tool for various computer vision applications. The model uses an advanced C2f module to enhance feature extraction and contextual understanding, significantly improving detection accuracy. YOLOv8 also introduced YOLOv8-Seg, a semantic segmentation model that combines a CSPDarknet53 feature extractor with the C2f module, achieving state-of-the-art performance in multiple benchmarks while maintaining high efficiency in both object detection and segmentation tasks[49].

YOLOv9 introduced several groundbreaking techniques, including the Information Bottleneck Principle and Reversible Functions, which help preserve essential data across the network’s depth and improve model convergence and performance. The introduction of Programmable Gradient Information, a technique that selectively controls gradient signals during training to enhance learning from the most important features, enables the model to optimize learning

efficiency by focusing on the most informative gradients, which is especially useful for light-weight models prone to losing information. Additionally, GELAN (Gradient Enhanced Light-weight Architecture Network) optimizes computational efficiency, ensuring the model performs well even with fewer parameters [49].

YOLOv10 was developed by researchers at Tsinghua University, and it focused on enhancing the CSPNet backbone for improved gradient flow and reducing computational redundancy. The architecture includes a Path Aggregation Network for better feature fusion across multiple scales, making it more efficient for detecting objects of various sizes. YOLOv10 also introduced NMS-Free Training, which eliminates the usual step of removing overlapping boxes, making detection faster and simpler without losing accuracy by reducing latency and streamlining the inference process[49].

YOLOv11 represents a significant advancement over its predecessors, with several architectural innovations aimed at optimizing both accuracy and efficiency. This version incorporates new techniques to enhance feature extraction, multi-scale detection, spatial attention, and computational efficiency [49].

Backbone and Feature Extraction Enhancements

The backbone of YOLOv11 plays a critical role in efficiently extracting meaningful features from input images. A primary innovation in the backbone architecture is the introduction of the C3K2 block, which builds upon the Cross-Stage Partial structure. This new block uses smaller 3x3 kernels in place of larger kernels, enabling faster computations without compromising the model’s ability to capture essential image features. By using smaller convolutions, YOLOv11 improves computational efficiency while still preserving the quality of extracted features. This change represents a shift towards using less computationally expensive operations to achieve better performance, thereby optimizing the model for real-time object detection applications.

In addition to the C3K2 block, YOLOv11 continues to use the C2F (Cross-Stage Partial Focus) block, which splits the feature map into two halves and processes them separately through bottleneck layers, that reduce feature size to make processing faster and more efficient without losing important details,before concatenating them back together. This design reduces redundant information and enhances feature map preservation, ensuring that important features are not lost during processing. The combination of the C3K2 and C2F blocks in the backbone allows YOLOv11 to achieve a balance between speed and accuracy, offering a more efficient method of feature extraction compared to earlier versions.

Bottleneck Structure and Shortcut Connections

YOLOv11 further improves its architecture by incorporating bottleneck layers with shortcut connections, drawing inspiration from the ResNet architecture. Bottleneck structures, consisting of several convolutional layers, are made to help the model understand important details using fewer calculations.The shortcut connections in YOLOv11 allow for residual paths that bypass certain convolutional operations, ensuring the smoother propagation of gradients during the backpropagation process. This mechanism prevents the vanishing gradient problem, particularly in deeper networks, and accelerates the training process.

The presence of shortcut connections allows YOLOv11 to maintain performance even as the network grows deeper. This is crucial for learning complex patterns and representations, es-

pecially when processing large amounts of data. These improvements in the bottleneck structure contribute to more efficient learning, faster convergence, and ultimately, better generalization during the detection of objects in real-world applications.

Spatial Pyramid Pooling Fast for Multi-Scale Detection

One of the most significant innovations in YOLOv11 is the integration of the Spatial Pyramid Pooling Fast (SPFF) module, which addresses the challenge of detecting objects at different scales. Traditional object detection networks often struggle with detecting small objects, especially when they occupy a limited portion of the image. The SPFF module solves this problem by performing pooling operations at multiple scales, aggregating contextual information across various resolutions. This enables YOLOv11 to effectively handle objects of varying sizes, improving its ability to detect small and medium-sized objects without sacrificing the detection of larger objects.

The SPFF module achieves this by applying max-pooling operations with different kernel sizes. These pooling operations aggregate multi-scale information, allowing the network to extract relevant features from different regions of the image. The inclusion of SPFF ensures that YOLOv11 can detect small objects that might have been overlooked by earlier versions of YOLO, making it particularly well-suited for applications that require high accuracy in detecting objects of various sizes.

Attention Mechanisms: C2PSA for Spatial Focus

YOLOv11 introduces attention mechanisms through the C2PSA (Cross-Stage Partial with Spatial Attention) block, a key innovation designed to improve the model’s ability to focus on the most relevant regions of an image. The C2PSA block uses two Partial Spatial Attention modules that process different branches of the feature map independently before concatenating them. This spatial attention mechanism enables the model to emphasize regions of the image that are more likely to contain important features for object detection, such as small, occluded, or partially visible objects.

The use of spatial attention allows YOLOv11 to better focus on finer details within an image, improving its ability to detect objects in challenging scenarios where parts of the objects may be missing or obscured. This innovation is particularly beneficial when detecting objects in complex environments, as it helps the model prioritize the most relevant features for accurate detection. The spatial attention mechanism enhances the overall robustness of YOLOv11 in scenarios where previous YOLO versions might have struggled.

Position-Sensitive Attention for Precise Feature Extraction

In addition to the C2PSA block, YOLOv11 also incorporates Position-Sensitive Attention, which refines the model’s ability to focus on important spatial features. This mechanism processes the feature map through dedicated layers designed to capture the spatial relevance of different regions before applying the attention mechanism. By focusing on the positions of important objects within the image, the model can selectively enhance the features that are most relevant for accurate detection.

Position-sensitive attention helps YOLOv11 adjust its attention dynamically based on the spatial location of objects, improving the precision of object detection. This attention mechanism is particularly effective in complex scenes where the relative positions of objects play a crucial role in determining their identity and location. By enhancing the model’s ability to focus on

specific areas of interest, position-sensitive attention contributes to YOLOv11’s ability to detect and classify objects with greater accuracy, especially in cluttered or dynamic environments.

Multi-Scale Detection Head for Better Object Size Handling

YOLOv11 further improves its object detection capabilities by refining its multi-scale detection head. The model makes predictions at three different scales: low, medium, and high. These correspond to feature maps at varying levels of granularity (P3, P4, and P5), with finer feature maps (P3) used to detect small objects and coarser maps (P5) used to detect larger objects. By using multiple scales to make predictions, YOLOv11 can detect objects of varying sizes with greater precision and accuracy. P3 has more detail for small objects, while P5 is less detailed but better for big objects.

This multi-scale detection approach ensures that both small and large objects are detected effectively, without the risk of missing smaller objects or overfitting to larger ones. By incorporating predictions from multiple feature maps, YOLOv11 offers a more flexible and comprehensive approach to object detection, which is essential for real-time applications where the presence of objects at various scales is common.

Efficiency and Real-Time Performance

Despite the numerous innovations and improvements in YOLOv11, it maintains the hallmark of YOLO models: real-time performance. The integration of smaller convolutions in the C3K2 block, the optimized SPFF and C2PSA attention mechanisms, and the multi-scale detection head all contribute to improving the model’s speed and computational efficiency. YOLOv11 is designed to achieve high accuracy in object detection without a significant increase in processing time, ensuring that it can perform in real-time applications like autonomous driving, video surveillance, and robotics.

The combination of these architectural innovations allows YOLOv11 to maintain its reputation for fast inference times while improving detection accuracy, particularly in challenging scenarios involving small or occluded objects. The ability to perform real-time detection with high accuracy makes YOLOv11 an ideal solution for a wide range of applications where speed and precision are critical.

2.6.6.1 Comparison of YOLO with CNNs, Object Detection and Image classification models

YOLO combines all object detection components into a single neural network, processing the entire image in one pass to predict bounding boxes and class labels simultaneously. The image is divided into an S × S grid, with each cell detecting objects whose center falls within it. Each cell predicts multiple bounding boxes, confidence scores (reflecting object presence and box accuracy), and class probabilities. The confidence score combines the object's presence probability with the IOU between predicted and actual boxes. These scores are multiplied by class probabilities to generate class-specific confidence scores, enabling real-time, high-precision object detection and end-to-end training[47].

For example, imagine a photo divided into a 7x7 grid. If a dog’s head is mainly inside one grid cell, that cell is responsible for detecting the dog, predicting where its bounding box should be, and guessing that the object is a dog with a certain confidence score. This way, YOLO quickly and accurately finds objects in the whole image.

As a real-time, single-stage object detection model, YOLO excels in detecting and localizing various objects or behaviors simultaneously, making it ideal for applications such as driver monitoring systems. YOLO can detect multiple distractions, such as phone use, eye closure, and head movement, in a single frame. Unlike traditional CNN-based models, which are typically used for feature extraction but not localize distractions, YOLO not only identifies the presence of these distractions but also pinpoints their location within the image. This dual capability of detection and localization sets YOLO apart from CNNs, which, while effective at classifying an image as "distracted" or "focused," cannot provide the level of detail or real-time feedback needed for monitoring multiple behaviors at once.

In contrast, Faster R-CNN is a two-stage model that first generates region proposals and then classifies them. While two-stage models like Faster R-CNN tend to offer higher accuracy, their additional computational steps make them slower and therefore making them less suitable for tasks where rapid feedback is essential.

A key strength of YOLO is its adaptability. The model has demonstrated impressive generalization capabilities, performing well on diverse datasets even if they were not part of its training set. For instance, when trained on natural images and tested on artwork, YOLO outperformed other detection systems, highlighting its ability to adapt to different contexts and environments. This flexibility makes YOLO a reliable and versatile tool in various object detection scenarios, including driver monitoring[47].

2.6.7 **Methods for improving ML techniques**

Regularization techniques are essential for addressing the challenge of overfitting in ML models, where a model performs exceptionally well on training data but poorly on unseen test data. Overfitting occurs when a model memorizes training data instead of generalizing, while underfitting happens when it fails to learn adequately. Techniques such as dropout and drop-weights help mitigate overfitting by randomly removing neurons or their connections during training, forcing the network to learn more robust and independent features. Data augmentation artificially increases the size of the training dataset, enhancing model generalization. Another critical method is batch normalization, which normalizes activations in each layer to reduce internal covariance shifts and stabilize training. Batch normalization also accelerates convergence, prevents vanishing gradients, and minimizes sensitivity to hyperparameter tuning, making it a powerful tool for improving model performance and reducing overfitting [38].

Regularization

Optimizer selection is a critical aspect of training CNNs, as it directly influences the learning process and model performance. Optimizers are algorithms designed to adjust the network's parameters, such as weights and biases, to minimize prediction errors. Gradient-based learning, the most common optimization method, iteratively updates these parameters by calculating gradients and adjusting them in the direction that reduces the loss function. There are several types of gradient descent optimizers, including batch gradient descent, stochastic gradient descent, and mini-batch gradient descent, each with unique advantages in terms of speed, stability, and memory usage. Techniques like momentum and Adaptive Moment Estimation (Adam) further enhance these algorithms by improving convergence speed and stability. Momentum helps smooth weight updates by considering gradients from previous steps, while Adam combines the benefits of momentum and RMSprop, adapting learning rates for

each parameter for efficient training. Proper optimizer selection ensures effective learning, balances resource utilization, and addresses challenges such as local minima and noisy gradients [38].

Improving the performance of CNNs can be achieved through various methods. Expanding the dataset with data augmentation or leveraging transfer learning enables the model to generalize more effectively. Allowing more training time helps the network to better refine its parameters and enhance accuracy. Increasing the depth or width of the model’s architecture boosts its ability to capture intricate patterns and features. Applying regularization techniques mitigates overfitting and strengthens the model's generalization capabilities. Additionally, optimizing hyperparameters such as the learning rate, batch size, and activation functions ensures the network is well-tuned for the task, leading to improved results [38].

There are also other techniques, such as Long Short-Term Memory networks, which address the vanishing gradient problem in Recurrent Neural Networks. A Long Short-Term Memory unit includes memory cells and three gates, input, output, and forget, that control data storage, reading, and erasure. The input gate decides what new information to add, the forget gate removes old or unnecessary information, and the output gate controls what memory information is passed on to the next step. Gated Recurrent Units offer a simpler alternative by using a single hidden state and two gates: the update gate for retaining information and the reset gate for controlling the interaction with the current input. Transfer learning is another technique, where a model trained for one task is reused for a related task. In image classification, network-based transfer learning is common, where a pretrained model is fine-tuned for a new task, typically by replacing its classification layer and adjusting it to the target dataset[18].

Transfer learning is an ML technique in which a model trained on one task is reused for a related task. It involves transferring knowledge from one domain to another, especially when the target dataset is smaller than the original dataset. There are various approaches to transfer learning, including instance-based, mapping-based, network-based, and adversarial-based methods. In image classification, the network-based approach is commonly used, where a pretrained model is adapted for a new task by replacing its classification layer and fine-tuning the model's weights for the target dataset. This technique helps improve performance by utilizing features learned from the original task [18].

2.6.8 Algorithms evaluation

In ML, the effectiveness of an algorithm is typically evaluated using a core set of performance metrics. In the context of distraction detection, most studies focus on classification problems, where the goal is to determine whether a driver is distracted. These classification problems can be divided into different types: binary classification (distracted vs. non-distracted driver), multiclass classification (such as smoking, eating, or reaching for something), and multi-labelled classification (where multiple actions are happening simultaneously in a single scene). In a typical classification task, evaluation metrics are used in two key stages: the training stage and the testing stage. During the training stage, the evaluation metric helps optimize the classification algorithm, ensuring it learns to distinguish between the different classes effectively. During the testing stage, the evaluation metric measures the performance of the model by testing it on unseen data, which provides insight into how well the classifier generalizes to new, real-world situations [18].

For binary classification problems, such as determining if a driver is distracted or not, the evaluation is often based on a confusion matrix. The confusion matrix is a tool that compares the predicted classifications with the actual outcomes. It consists of rows representing the predicted classes and columns representing the true classes. This matrix helps to assess the performance of the model by calculating various metrics derived from the values in the matrix. Common metrics include accuracy, precision, recall, and F1 score, each measuring different aspects of the model's performance. These metrics are essential for evaluating a model's ability to detect distractions in real-time driving scenarios, and some of them are extended to handling multiclass problems[18].

Accuracy is the most used as evaluation metric for both binary and multiclass classification problems, as it measures the percentage of correct predictions out of the total instances. However, it is most effective when the class distribution is balanced, which is rarely the case in real-world scenarios. When class distributions are imbalanced, the F1-Score is a more favorable metric. The F1-Score combines precision and recall into a single measure, offering a more balanced evaluation of the model's performance and ensuring that both classes are properly identified, making it a more reliable metric for distraction detection algorithms[18].

The Receiver Operating Characteristic curve is used to evaluate how well a classification model performs at different thresholds by plotting two factors: the true positive rate and the false positive rate. The area under the curve is a key metric that shows how well the model can distinguish between classes. A good model will have a high area under the curve, while a poor model will have a value close to 0.5, meaning it cannot separate the classes. For problems with multiple classes, the curve can be extended by comparing one class to all the others [18].

Loss functions are used in ML to reduce the difference between the model's predictions and the actual labels, improving its accuracy. Common loss functions include Cross-Entropy loss, which is used for models that predict probabilities, and Triplet loss, which helps the model focus on correctly distinguishing similar and different samples. Other loss functions, such as 0-1 loss and Hinge loss, are also used in distraction detection tasks. These loss functions help improve the model's ability to detect and classify distractions based on data from sensors [18].

2.7 Monitoring Techniques and Action Classification

There are various methods and perspectives for detecting whether a driver is distracted, which can be broadly categorized into driver monitoring methods and action classification techniques.

In driver monitoring, distractions are identified by observing specific movements or positions of the driver's body parts, such as the eye gaze, head pose, hand position, and overall body posture. These physical cues are used as indicators of the driver's level of attention. For instance, if a driver's eyelids are closed for an extended period, it can be inferred that the driver is experiencing fatigue. Similarly, the detection of hand movements plays a key role; if both hands are off the steering wheel, it might indicate that the driver is engaging in non-driving-related tasks such as using a phone or reaching for an object.

Head pose and body posture also provide crucial insights. For example, if the driver's head is turned away from the road for a prolonged time, it could signal that the driver is looking at

something unrelated to driving, such as a passenger or a device. This method relies on monitoring body language and physical behavior to assess whether the driver is focused on the road.

Action classification, on the other hand, involves detecting specific predefined distracting activities. These actions might include behaviors such as reaching behind, drinking, eating, or using a phone. In this approach, datasets of images and/or videos showcasing these distraction activities are collected. These datasets are then processed and analyzed using ML models to classify the specific distracting actions. By training these models on a wide range of examples, they learn to recognize patterns associated with distractions.

Action classification methods often utilize publicly available datasets for training and evaluation. For instance, the StateFarm dataset contains annotated images of drivers engaging in these various activities, while the AUC Distracted Driver Dataset provides video sequences and images specifically designed for distracted driving detection[50], [51], [52]. These datasets help researchers standardize their methods and improve the performance of their models by enabling comparisons across different approaches[18].

In earlier studies focusing on driver monitoring methods, much of the research primarily concentrated on identifying cellphone usage as a key form of driver distraction. This was typically done by using images captured from frontal cameras positioned to monitor the driver. These images were then processed through various steps, beginning with cropping specific regions of interest, followed by isolating the pixels corresponding to the driver’s skin. This process helped to locate the position of the hand, particularly when it was near the face region, which could indicate cellphone use. Beyond cellphone detection, other studies explored a broader range of facial features to infer driver distraction. These included analyzing specific parts of the face, such as the head, eyes, mouth, lips, and eyebrows. By focusing on these features, the studies aimed to provide a more nuanced understanding of the driver’s state and behavior. These methods were particularly useful in identifying subtle distractions that might not involve obvious movements like handling a phone [18].

In addition to facial feature analysis, some studies incorporated the use of head pose estimation. This technique was employed to further improve the accuracy of detecting distractions by analyzing the orientation and movement of the driver’s head. Head pose estimation provided an additional layer of information that could help differentiate between focused and distracted behavior, thus enhancing the reliability of the overall monitoring system [18].

Ali and Hassan [53] proposed a driver monitoring method based on facial features, specifically using motion vectors and interpolation to detect driver distraction caused by head rotation due to changes in the yaw angle. These facial points were detected using the Active Shape Model and Boosted Regression with Markov Networks. The approach relied on a single camera mounted on the car's dashboard to capture images of the driver. The authors trained and tested various classifiers and found that the approach using motion vectors and interpolation outperformed other methods in detecting head rotation. They achieved an accuracy of 98.45% using a Neural Network classifier, demonstrating the effectiveness of their monitoring method in detecting head movement as a form of driver distraction [18].

Traditional ML methods, useful in some cases, did not achieve high levels of accuracy. One of the key limitations of these methods is their inability to capture and process crucial features

that may indicate driver distraction. As a result, more recent studies have turned to CNNs and DL techniques, which are better equipped to handle complex data and extract more meaningful features from it. These advanced methods are particularly beneficial in the field of detection of driver distraction, as they can analyze and learn patterns from large datasets, improving accuracy over traditional ML approaches[18].

For instance, Choi et al. [54] presents a study in which the driver's gaze zone is categorized using new DL techniques. The sequence of gaze zones of a driver reflects precisely what and how the driver behaves, which allows us to infer their drowsiness, focus, or distraction by analyzing images captured from a camera. A Haar feature-based face detector is combined with a correlation filter-based MOSS tracker for the face detection task to handle the tough visual environment inside the car. The driving database used in this study is a large dataset constructed using a recording setup within a compact sedan, with the car being driven around urban areas. The gaze zones consist of 9 categories, such as straight ahead, left mirror, right mirror, dashboard,depending on where the driver is looking while driving. A CNN is trained to categorize the driver's gaze zone from a given face-detected image using a multi-GPU platform. After training, the network parameters are transferred to a GPU within a PC running Windows to operate in real-time. The results suggest that the accuracy of gaze zone categorization reaches 95% on average, indicating that the system outperforms state-of-the-art gaze zone categorization methods based on conventional CV techniques.

Furthermore, DL techniques are also used to estimate the driver’s head pose, which can provide additional valuable information for distraction detection. The ability to accurately determine the head pose allows the system to assess the driver’s attention and focus, as changes in head orientation can indicate distraction. By using CNNs and DL, these methods are able to learn and adapt to the subtle cues that may indicate driver inattention, leading to more robust and accurate distraction detection systems [18].

A common way to detect driver distractions involves analyzing their actions rather than focusing solely on specific body parts. This broader approach observes behaviors to identify distractions and offers more flexibility compared to methods that depend on individual characteristics or specific movements. Images are often used to study these actions, captured through techniques like standard photography, infrared imaging, or thermal imaging, and taken from different angles, such as from the front or side. These images are typically designed to show common distractions, such as drinking, smoking, or reaching for objects, which are frequent causes of lost attention while driving.

This area of study can be divided into two main types based on how datasets are used. In some cases, datasets are created for private use, while in others, publicly available datasets are utilized to classify driver behaviors. Many approaches treat distraction detection as a way of categorizing images into specific types of actions, using advanced artificial intelligence tools. Popular DL models are often applied to process these images, helping to accurately identify distracting behaviors. These methods contribute to the development of technologies that improve driver safety by monitoring and addressing distractions effectively[18].

2.8 Previous Work

YOLO models are used for real-time detection of driver distractions, such as phone use, fatigue, and drowsiness. Researchers have proposed modifications, including dilated convolutions, attention mechanisms, and lightweight designs, to enhance accuracy and performance, especially for embedded systems. These improvements boost detection accuracy and system efficiency, offering promising solutions to reduce road accidents caused by driver inattention. This section reviews recent studies on these advancements in YOLO models.

Chang et al. (2020) introduced the YOLOv3-tiny-dilated network, an improved version of YOLOv3 designed for speed and efficiency. It is lighter, making it suitable for mobile and embedded systems, though one-stage networks usually have slightly lower accuracy. To improve this, they addressed scale variation by replacing the original Feature Pyramid Network with three dilated convolution branches. This change resulted in a 2% increase in accuracy on the COCO dataset and a 95.47% accuracy in detecting driving distraction behavior[55].

Poon et al. (2021) developed a non-contact driving behavior detection system using YOLO-based models and a dashboard webcam. The models (YOLOv3-tiny, YOLOv3-tiny-3l, YOLO-fastest, and YOLO-fastest-xl) were trained to detect behaviors such as normal driving, distracted head turning, drowsiness, eating, and phone use. YOLO-fastest-xl gave the best results with an F1-score of 91.84%, FNR of 6.94%, and mean average precision (mAP) of 95.81%, running at 30 FPS on an NVIDIA GPU[56].

Sajid et al. (2021) tackled the issue of road accidents caused by distracted driving using the State Farm distracted driver dataset, which includes eight distraction categories. They used the EfficientDet model and compared it to Faster R-CNN and YOLOv3. EfficientDet-D3 achieved the best performance with a mAP of 99.16%, demonstrating its effectiveness in detecting distracted drivers and encouraging safer driving habits[57].

Ataş et al. (2021) focused on detecting driver distraction caused by mobile phone use and smoking. Using images from a mobile phone mounted on the windshield and a YOLOv5s network trained on labeled images, they successfully differentiated between drivers and passengers. The study also aimed to improve detection accuracy by adding classes like 'Driver Hand,' 'Passenger Hand,' and 'Driver Hand With Phone' to better detect various in-vehicle distractions[58]

Wei et al. (2022) proposed an improved DL algorithm based on the YOLOX framework to detect changes in driving behavior in real time. The model integrates a Convolutional Block Attention Module at multiple scales for better feature extraction. Training was done using the State Farm Distracted Driver Detection Dataset. Results showed that the model outperformed the original YOLOX framework, achieving higher mean average precision and reduced inference time, proving its effectiveness in detecting distracted driving[59].

Wang et al. (2023) proposed an improved YOLOv5-based method to detect driver attention states, focusing on fatigue and distractions. Fatigue is identified by calculating the aspect ratio of the eyes and mouth to detect closed eyes and yawning. For distraction detection, the YOLOv5 model is enhanced with Swin Transformer modules and modified feature extraction networks. Trained on datasets of distractions like smoking, drinking, and phone use, the model achieved a 2.4% mAP improvement, effectively identifying fatigue and distractions to improve driving safety[60].

Zheng et al. (2023) proposed a fatigue driving detection model based on DL, balancing real-time performance and accuracy. The model integrates modules for object detection, head pose estimation, fatigue, and distraction detection. Using YOLOv5 and FSA-Net for facial feature detection, it identifies the driver’s eye, mouth, and head states, as well as cognitive and visual distractions. The system includes closed-eye detection, yawning detection, and 3D head pose estimation, providing timely alerts to enhance driving safety with high accuracy[61].

Murali et al. (2024) used the YOLO algorithm to develop a real-time driver fatigue monitoring system using onboard camera footage. The system detects facial and eye features to identify fatigue indicators like drowsy gaze, blink rate, and eye closure. Trained on a large fatigue dataset, it achieves high accuracy and low false-positive rates. The system alerts drivers when drowsiness is detected, providing a reliable solution to improve road safety and reduce fatigue-related accidents[62].

Ma et al. (2024) introduced an improved YOLOv8 method for detecting distracted driving and driver emotions. The model combines a multi-head self-attention mechanism for distraction detection with a CNN module for emotion recognition. Trained on the FER2013 and custom datasets, it outperforms standard YOLO methods. Optimized for real-time use on a Jetson Nano platform with TensorRT and DeepStream, the improved YOLOv8 achieves better accuracy and speed, proving its effectiveness for real-world applications[63].

3 Methodology

This chapter includes a review of existing literature, dataset preparation, model training, and evaluation metrics.

3.1 Literature Search

The literature search focused on identifying studies related to three main criteria. First, it prioritized research that directly addressed driver distraction and its theoretical background. Second, it examined studies on distraction detection using machine learning, particularly those utilizing real time methods and the YOLO architecture. Lastly, the search favored peer-reviewed journal articles, conference proceedings, and other authoritative academic sources, such as books and reports, to ensure the credibility and reliability of the information.

he literature search was primarily conducted using Google Scholar, a platform that provides access to academic articles across various disciplines. This facilitated the identification of relevant studies on driver distraction detection, as well as ML and DL-based systems. Additionally, it enabled access to peer-reviewed journal articles and conference proceedings, ensuring the credibility and reliability of the sources.

The search was conducted systematically using Boolean operators and relevant keywords:

- “Introduction to driver distraction”
- "Driver distraction detection using YOLO"
- "DL for distracted driving"
- "ML for distracted driving"

Both foundational research and recent advancements were reviewed to develop a comprehensive understanding of the field.

3.2 Dataset Description

This research utilizes a dataset of 1,380 annotated images to train and evaluate the performance of the YOLOv11 object detection model for detecting driver distractions. Sourced from Roboflow, the dataset is specifically designed to capture a variety of distractions and behaviors within a vehicle environment, with an emphasis on driver safety. It includes a broad range of activities that could potentially distract drivers, such as Drinking, Hair and Makeup, Operating the Radio, Reaching Behind, Safe Driving, Talking on the Phone, Talking to Passenger, Texting.

The dataset is organized for object detection tasks, with each image containing bounding box annotations that pinpoint the locations of specific driver actions or distractions. These annotations are essential for identifying both the position and behavior of the driver or passengers in real-time driving scenarios.

3.2.1 **Dataset Breakdown**

The dataset is divided into three subsets to facilitate model training, validation, and testing. It contains a total of 1,380 images, with 969 images (70.3% of the total) allocated to the training set, 274 images (19.9%) designated for validation, and 137 images (9.9%) reserved for testing.

Each image is annotated with bounding boxes that highlight the areas of interest where distractions or driver behaviors are occurring. These annotations span 8 distinct classes each

representing a different form of driver distraction or activity. These behaviors include both distracting actions (like texting or talking on the phone) and safer behaviors (like attentive driving).

3.2.2 Annotation Format and Object Detection Task

Annotations in the dataset follow the YOLO format, which includes the coordinates of each bounding box (center coordinates, width, and height) along with the class label for each detected activity (e.g., texting, talking on the phone). The YOLOv11 model, like its predecessors, is designed to predict both the location and the class of each detected activity within an image.

This structure enables the model to simultaneously detect and localize multiple distractions or behaviors in an image.

3.2.3 Image Characteristics

All images are 640x480 pixels, ensuring that even small distractions (e.g., texting or adjusting the radio) can be clearly detected while maintaining processing efficiency for the YOLOv11 model. The dataset uses a 4:3 aspect ratio, similar to the field of view captured by in-car cameras or dash cams, which enables the model to detect behaviors across a broad context within the vehicle.

With an average size of approximately 0.31 megapixels per image, this resolution strikes a balance between detail and computational efficiency, allowing the model to handle large datasets without excessive resource demands. Larger images would slow processing, while smaller ones could reduce detection accuracy.

3.3 Data partitioning and model configuration

After the data preprocessing, the next step was partitioning the data. The annotated dataset was split into three main sets: a training set, a validation set, and test set. The training set was used to train the model, while the validation set was used to evaluate its performance during the training process. This partitioning ensures that the model can learn from a substantial amount of data while also being able to assess its accuracy on a separate validation set, helping to prevent overfitting and ensuring generalization to unseen data.

To prepare for model training, a configuration file in the ".yaml" format was created after the data partitioning. This configuration file specified the directories for both the training and validation data, as well as the number of object classes and their corresponding names. The .yaml file is a simple, format that uses key–value pairs to organize configuration settings. In the context of YOLO, it tells the model where the data is located, how many classes to expect, and what each class represents, ensuring the training process is correctly aligned with the dataset structure.

3.4 Installing requirements and starting training the models

To begin training the model, the YOLOv11 object detection algorithm was set up using the Ultralytics package. First, the latest version of the Ultralytics package was installed via pip. After installation, the YOLOv11 algorithm, along with necessary tools from the IPython package (like the display and Image classes), were imported for use during training.

The training process was then started by running the YOLOv11 object detection algorithm with specific parameters, such as the detection task, training mode, model file, data file, number of epochs, image size, and plot settings. The model was trained using a custom ".yaml" data file

for 55 epochs, one complete iteration over the entire training dataset during the training process, with an image size of 640 pixels to ensure optimal performance.

3.5 Model Training

The YOLOv11 algorithm from the Ultralytics library was imported. To begin the object detection process, the "predict" method was called on the YOLO object. Additionally, the "conf" argument was set to 0.8 (80%), establishing the minimum confidence threshold for object detection. This means that the model would only consider detections with a confidence score above 80%. The output of the "predict" method was stored in the "result" variable.

3.6 Evaluation Metrics and Statistical Analysis

This section discusses the methods employed to evaluate the performance of the trained model using a confusion matrix, statistical metrics.

3.6.1 **Confusion Matrix**

The confusion matrix serves as a foundational tool for analyzing classification performance, especially in classification tasks. It provides a detailed breakdown of the model's predictions by displaying the counts of true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN). TP refers to instances that are correctly predicted as positive, while TN are instances that are correctly predicted as negative. FP are instances that are incorrectly predicted as positive, and FN are instances that are incorrectly predicted as negative. These four quantities are essential for calculating many of the performance metrics, such as Precision, Recall, and F1-Score.

The confusion matrix provides a comprehensive view of how well the model is distinguished between the classes. It also allows for the calculation of important metrics that assess the model's ability to identify positive and negative instances correctly. By analyzing the confusion matrix, it can identified whether the model has a bias towards one class.

In cases where the dataset is imbalanced, the confusion matrix helps to highlight the shortcomings of the model. For example, if the model has many FN or FP, the confusion matrix can point to areas where the model needs improvement.

Since the classification task involves 8 distinct categories, a confusion matrix tailored for multiclass problems is required. In a multiclass classification setting, with N unique class labels (Ci, where i = 1, 2, ..., N), the confusion matrix is organized as an N × N grid. Each column in the matrix represents the predicted class, while each row represents the actual class. The values in the matrix reflect the number of instances where the predicted class corresponds to the actual class. Specifically, the cell at the intersection of row 'i' and column 'j' indicates the number of instances where the predicted class is 'j' and the true class is 'i'.

Table 5 illustrates the structure of a multi-class classification confusion matrix.

Table 5: The structure of a multi-class classification confusion matrix

		Predicted class			
		C_1	C_2	...	C_N
Actual class	C_1	$C_{(1,1)}$	FP	...	$C_{(1,N)}$
	C_2	FN	TP	...	FN

	⋮	⋮	⋮	⋮	⋮
	C_N	$C_{(N,1)}$	FP	…	$C_{(N,N)}$

TP (True Positive): Correctly detects a distraction.
TN (True Negative): Correctly identify no distraction.
FP (False Positive): Incorrectly detects a distraction when none exists.
FN (False Negative): Misses detecting an actual distraction

3.6.2 Evaluation Metrics for Model Performance

In the evaluation of classification models, it is essential to use a set of metrics that provide a comprehensive understanding of the model's performance across various aspects. Key metrics include Accuracy, Precision, Recall, and F1-Score, each of which can be calculated individually for each class in the dataset. Additionally, macro and micro averages of Precision, Recall, and F1-Score are calculated to provide a broader view of the model’s behavior. This paper discusses the significance of these metrics and why they are crucial in assessing the results of a classification model.

Accuracy

Accuracy is one of the most straightforward and widely used metrics for classification performance. It is defined as the ratio of correctly predicted instances to the total number of instances. While accuracy provides an overall measure of the model's effectiveness, it can be misleading when the dataset is imbalanced. In cases where one class significantly outnumbers another, a model could achieve high accuracy by predicting the majority class correctly while performing poorly on the minority class. Therefore, while accuracy is an important metric, it must be interpreted in conjunction with other performance measures.

Precision

Precision evaluates the proportion of true positive predictions (correctly identified instances of a class) among all the instances predicted as belonging to a particular class. It is a critical metric when the cost of FP is high. For example, in driver distraction detection, false positives can annoy or distract the driver unnecessarily, cause them to lose trust in the system, and lead to ignoring future important alerts. Precision ensures that when the model makes a positive prediction, it is likely to be correct, making it an essential metric for use cases where avoiding FP is paramount.

Recall

Recall, or sensitivity, measures the ability of the model to correctly identify all instances of a particular class. It is the proportion of true positive predictions among all actual instances of that class. Recall is especially important in contexts where FN are costly or dangerous. For instance, in driver distraction detection task, failing to detect a distraction (false negative) can have serious consequences. A high recall ensures that as many true instances of a class as possible are correctly identified, which is crucial in situations where missing a positive instance can lead to harm.

F1-Score

The F1-Score is the harmonic mean of Precision and Recall, providing a balanced metric that accounts for both FP and false negatives. It is particularly valuable in scenarios where there is an imbalance between Precision and Recall. A high F1-Score indicates that the model maintains a good balance between Precision and Recall, which is often the desired outcome in

many practical applications. In cases where Precision and Recall are inversely related (i.e., improving one decreases the other), the F1-Score offers a more informative evaluation than Precision or Recall alone. It is especially useful when working with imbalanced datasets, where one class might dominate, and both FPs and FN need to be carefully controlled.

Macro-Averaging

Macro-averaging calculates the metric (Precision, Recall, or F1-Score) independently for each class and then averages them. This method treats all classes equally, regardless of their size or the number of instances they contain. Macro averages are useful when the goal is to assess the model's performance across all classes equally, giving equal weight to each class in the evaluation process. This is particularly important in situations where the model must perform well in all classes, not just the majority class. Macro averages are often used in imbalanced datasets to ensure that the model does not disproportionately favor the majority class.

Micro-Averaging

In contrast, micro-averaging aggregates the contributions of all classes by summing the TP, FP, and FN across all classes before calculating the metric. Micro averages provide an overall assessment of the model's performance by taking into account the total number of TP, FP, and FN. This method is particularly useful when the goal is to evaluate the model's overall performance across all classes, with more weight given to the classes with more instances. Micro averages are especially beneficial in datasets where one class is dominant, as they reflect the model's performance on the entire dataset rather than on individual classes.

The combination of Accuracy, Precision, Recall, F1-Score, and their macro and micro averages provides a well-rounded evaluation of a classification model. Each metric serves a unique purpose in capturing different aspects of model performance. Accuracy offers a general overview of the model's performance, though it is less useful in the context of imbalanced datasets. Precision and Recall give more specific insights into how the model handles FPs and false negatives, respectively. F1-Score strikes a balance between these two, making it an ideal metric in cases where a trade-off between Precision and Recall is necessary.

The use of macro and micro averages ensures a more thorough evaluation of the model. Macro averages highlight the model's performance across all classes, treating each class equally, which is crucial for applications where the model must perform well on all classes. Micro averages, on the other hand, provide insight into the model's overall performance across all instances in the dataset, with more emphasis on the classes with more data. Both averaging methods offer complementary perspectives on model performance, ensuring that the evaluation is not biased towards the majority class or specific instances.

The following formulas were used to calculate these metrics[64]:

$$Accuracy = \frac{\sum_{i=1}^{N} \mathrm{TP}\ (C_i)}{\sum_{i=1}^{N}\sum_{j=1}^{N} C_{i,j}} \tag{6}$$

$$Recall\ of\ class\ C_i = TPR(C_i)\ = \frac{TP\ (C_i)}{\mathrm{TP}\ (C_i) + FN\ (C_i)} \tag{7}$$

$$Precision\ of\ class\ C_i = PPV\ (C_i)\ = \frac{TP\ (C_i)}{TP\ (C_i) + FP\ (C_i)} \tag{8}$$

$$F1 - Score\ of\ class\ C_i = F_1(C_i)\ = \frac{2 * TPR(C_i) * PPV\ (C_i)}{TPR(C_i) + PPV\ (C_i)} \tag{9}$$

$$Recall\ (macro\ average) = TPR\ (macro) = \frac{1}{N}\sum_{i=1}^{N} TPR(C_i) \quad (10)$$

$$Precision\ (macro\ average) = PPV\ (macro) = \frac{1}{N}\sum_{i=1}^{N} PPV(C_i) \quad (11)$$

$$F1 - Score\ (\text{macro average}) = F_1(macro) = \frac{2 * TPR(macro) * PPV\ (macro)}{TPR(macro) + PPV\ (macro)} \quad (12)$$

$$Recall\ (micro\ average) = TPR\ (micro) = \frac{\sum_{i=1}^{N} TP\ (C_i)}{\sum_{i=1}^{N}[TP\ (C_i) + FN\ (C_i)]} \quad (13)$$

$$Precision\ (micro\ average) = PPV\ (micro) = \frac{\sum_{i=1}^{N} TP\ (C_i)}{\sum_{i=1}^{N}[TP\ (C_i) + FP\ (C_i)]} \quad (14)$$

$$F1 - Score\ (micro\ average) = F_1(micro) = \frac{2 * TPR(micro) * PPV\ (micro)}{TPR\ (micro) + PPV\ (micro)} \quad (15)$$

4 Results

In this chapter, the results of YOLOv11 are examined. First, the analysis of the training and validation phases is presented, followed by a discussion of the testing phase.

4.1 Training Results

Analysis of the changes in training and validation losses across multiple epochs, as presented in Table 6 and Figure 7, shows that the model demonstrates consistent improvements in both object localization and classification, as well as its ability to generalize to new data. Over time, both the training and validation losses decrease, accompanied by improvements in precision, recall, and mAP metrics, indicating effective learning. The gradual reduction in the learning rate suggests that the training process was carefully managed by model. Furthermore, the model's performance on the validation set is similar to its performance on the training set, indicating that it is not overfitting and it is capable of making accurate predictions on new data. However, a more detailed analysis of the losses, validation losses, and performance metrics is provided in the following.

Losses

The training losses, including box loss, classification loss, and DFL loss, show consistent improvement throughout the 50 epochs. The box loss, which measures how accurately the model predicts bounding boxes, starts at 1.02 and gradually decreases to 0.29 by epoch 50. In simple terms, a higher loss means worse performance with bigger prediction errors, while a lower loss reflects better performance and more accurate predictions. This suggests that the model is getting better at localizing objects over time.

Similarly, the classification loss, which evaluates the model’s ability to assign the correct class label to each detected object, drops from 3.44 in epoch 1 to 0.21 by epoch 50, indicating that the model is becoming more accurate in classifying the objects within the bounding boxes. Higher classification loss (like 3.44) means more errors at the beginning, while a lower value (like 0.21) indicates improved accuracy after training. The DFL loss also decreases from 1.59 to around 1.05, suggesting steady improvements in the model's ability to estimate distributions for detected objects. A higher DFL value (like 1.59) reflects less precise localization at the start, while a lower value (like 1.05) shows more accurate and confident positioning of objects as training progresses. Therefore, it suggests steady improvements in the model's ability to estimate distributions for detected objects.

Validation Losses

The validation losses follow a similar trend to the training losses, confirming that the model is generalizing well. The box loss on the validation set decreases from 0.91 in epoch 1 to 0.42 by epoch 50, indicating that the model is improving its ability to accurately locate objects in new, unseen images. A higher box loss means less precise bounding boxes early on, while a lower loss shows more accurate object localization after training. Similarly, the classification loss improves significantly, dropping from 4.61 in the first epoch to 0.32 by epoch 50, which means the model initially made many classification errors but greatly reduces them as it learns. The validation DFL loss also decreases, mirroring the training DFL loss trend, confirming steady improvement in precisely estimating object positions. A higher DFL loss reflects less

confidence and accuracy early in training, while the lower value shows more precise predictions later.

Metrics (Precision, Recall, mAP)

The validation losses follow a similar trend to the training losses, confirming that the model is generalizing well. The box loss on the validation set decreases from 0.91 at epoch 1 to 0.42 by epoch 50, meaning the model initially made relatively large errors when predicting object locations, but by the end of training, these errors were reduced by more than half, showing improved precision in bounding box predictions on new images. Similarly, the classification loss drops substantially from 4.61 at the start to 0.32 at the end, indicating that the model went from making many incorrect class predictions to making very few errors. The validation DFL loss also declines alongside the training DFL loss, from a higher value down to a more stable lower value, which shows the model's growing confidence and accuracy in estimating object boundaries throughout training.

Learning Rate

The learning rate follows a typical decay pattern. This gradual reduction allows the model to make large weight updates early in training for faster learning, then progressively smaller updates to fine-tune the model parameters. Lower learning rates in later epochs help prevent overshooting the optimal solution and stabilize training, thereby improving convergence and reducing the risk of overfitting. Such a decay schedule is widely adopted in DL to enhance model performance and generalization.

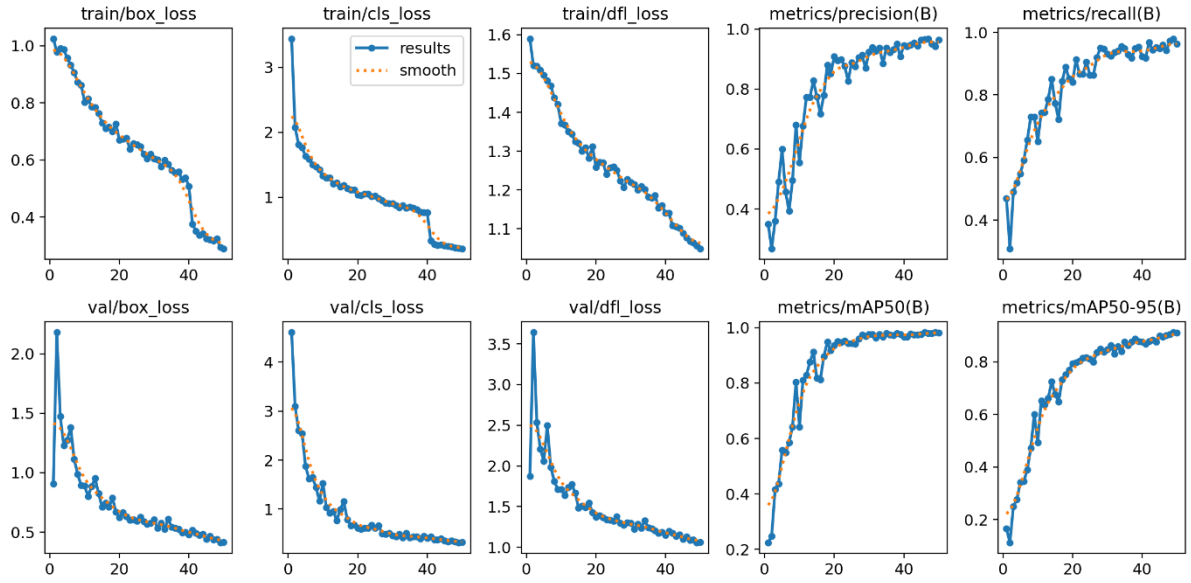


Figure 7: Training and validation losses, metrics, and learning rate across epochs

Table 6: Training and Validation Losses Across Epochs

Epoch	T-BL	T-CL	T-DL	V-BL	V-CL	V-DL
1	1.02486	3.44206	1.58997	0.90884	4.60852	1.87136
20	0.66979	1.04083	1.25837	0.62177	0.61529	1.37088
50	0.28953	0.20874	1.04839	0.41645	0.32441	1.06283

- T-BL: Train Box Loss
- T-CL: Train Classification Loss
- T-DL: Train DFL Loss

- V-BL: Validation Box Loss
- V-CL: Validation Classification Loss
- V-DL: Validation DFL Loss

4.2 Test results

In this section, the test results of the model across various distraction scenarios are presented. The model was evaluated using key metrics, including Precision, Recall, and F1-Score for each behavior class. The results, as shown in Table 8, reveal strong performance in certain behaviors, such as “Talking on the Phone”, while exhibiting challenges with classes that share some degree of similarity like “Safe Driving” and ” Talking to Passenger”. The confusion matrix in Table 7 provides further insights into these misclassifications, illustrating the frequency of FP and FN across the classes. In the following analysis, these results will be examined in greater detail to identify potential areas for model improvement.

Table 7: Confusion matrix

		True							
		C1	C2	C3	C4	C5	C6	C7	C8
predicted	C1	32							
	C2		23					1	1
	C3			26					
	C4				34				
	C5		1			29		2	
	C6	1	2				44		1
	C7					2		20	
	C8								53

C1: Drinking	**C5: Safe Driving**
C2: Hair and Makeup	**C6: Talking on the Phone**
C3: Operating the Radio	**C7: Talking to Passenger**
C4: Reaching Behind	**C8: texting**

Table 8: Class-wise classification metrics

Class	Precision	Recall	F1
Drinking	100,00%	96,97%	98,46%
Hair and Makeup	92,00%	88,46%	90,20%
Operating the Radio	100,00%	100%	100%
Reaching Behind	100,00%	100%	100%
Safe Driving	90,63%	93,55%	92,06%
Talking on the Phone	91,67%	100%	95,65%
Talking to Passenger	90,91%	86,96%	88,89%
Texting	100,00%	96,36%	98,15%

The "Drinking" class demonstrates exceptional classification performance, characterized by a perfect Precision of 1.0, indicating that every prediction the model made for this behavior was accurate with no false positives; in other words, the model never incorrectly labeled any other behavior as "Drinking," which is crucial for minimizing unnecessary alerts in practical applications. The Recall of approximately 0.97 signifies that the model correctly identified nearly all

actual instances of drinking, missing only one out of 33 true cases, resulting in a single false negative. This shows that while the model is highly sensitive to detecting the behavior, there is a very small chance of overlooking a drinking event. The combined metric, the F1-Score of 0.9846, reflects a strong balance between Precision and Recall, confirming that the model is not only accurate when it makes predictions but also reliably comprehensive in capturing true cases.

The "Hair and Makeup" class achieved a Precision of 0.92, indicating that 92% of the model's predictions for this class were correct, with a small proportion of false positives. The Recall was 0.88, meaning the model identified 88% of actual instances but missed 12%, resulting in some false negatives. Specifically, out of 26 true cases, 23 were correctly detected while 3 were missed. The F1-Score of 0.90 reflects the overall balanced performance of the model for this class, though it indicates there is some scope for improvement in detecting all true instances accurately.

The model achieves perfect classification performance for the "Operating the Radio" class, with Precision, Recall, and F1-Score all equal to 1.00. This indicates that the model correctly identified all true instances of this behavior without any false positives or false negatives, demonstrating complete accuracy and no misclassification in detecting this class.

The model demonstrates perfect classification performance for the "Reaching Behind" class, with Precision, Recall, and F1-Score all equal to 1.00. This means that all 34 instances of this behavior were correctly identified without any false positives or false negatives. The absence of false positives indicates that the model never confused other behaviors as "Reaching Behind," while the lack of false negatives means it detected every true occurrence. This flawless performance suggests the features and patterns associated with "Reaching Behind" are distinct and easily separable by the model, contributing to its high discriminative ability for this behavior.

For the "Safe Driving" class, the model achieved a Precision of 0.9063 and a Recall of 0.9355, with an F1-Score of 0.9206. The Precision indicates that about 90.6% of the model's predictions for safe driving were accurate, with 3 false positives where other behaviors were mistakenly labeled as safe driving. The Recall of approximately 93.6% means the model successfully identified most true safe driving instances but missed 2 cases (false negatives). This suggests that while the model is generally good at recognizing safe driving behavior, some actual safe driving moments may be misclassified, potentially due to subtle overlaps in appearance or motion with distracted behaviors like “Talking to Passenger”. The F1-Score reflects a balanced but not perfect detection capability.

The classification performance for the "Talking on the Phone" class shows a Precision of 0.9167 and a perfect Recall of 1.00. This indicates that while the model identified all 44 true instances of phone talking without missing any (no false negatives), it produced some false positives (4 cases), where other behaviors were incorrectly predicted as phone talking. The perfect Recall is critical for safety-critical applications, ensuring no instance of phone usage while driving is overlooked. The F1-Score of 0.9565 reflects strong overall performance with a slight trade-off between sensitivity and specificity.

For the "Talking to Passenger" class, the model achieved a Precision of 0.9091 and a Recall of 0.8696, resulting in an F1-Score of 0.8889. Precision here means that approximately 90.9%

of the model’s predictions were correct, with 2 false positives indicating some misclassification of other behaviors as talking to a passenger. The Recall of about 87% reveals that the model missed 3 true instances of this behavior. The relatively lower Recall compared to Precision suggests that some true "Talking to Passenger" events are difficult for the model to detect, possibly because the gestures and facial cues involved overlap with those in other classes such as "Safe Driving". This confusion reduces the model’s sensitivity in this category, and the F1-Score reflects the need for further improvement to enhance detection while maintaining accuracy.

The model’s performance on the "Texting" class is characterized by a perfect Precision of 1.00 and a high Recall of 0.9636, with an F1-Score of 0.9815. The perfect Precision indicates no false positives; when the model predicted texting behavior, it was always correct. The Recall value means the model correctly identified 53 out of 55 actual texting instances, missing 2 cases. The very high F1-Score suggests that the model is highly effective at both detecting texting behavior and avoiding incorrect classifications. The slight decrease in Recall may be due to variability in how drivers text or subtle differences in hand and head positions that occasionally make texting visually similar to other activities like operating the radio or reaching behind.

Table 9 presents the performance metrics of the classification model. The model achieves an overall accuracy of 98.99%, meaning it correctly classifies nearly 99 out of every 100 instances across all driver distraction categories and safe driving. This high accuracy shows that the model is effective at distinguishing between different driver behaviors in the dataset. However, accuracy alone may not fully reflect the model’s performance, especially in cases where some classes are much more common than others. Therefore, additional metrics such as Macro and Micro Precision, Recall, and F1-Score provide a more detailed assessment.

Macro-averaged metrics calculate precision, recall, and F1-score separately for each class and then take the average, giving equal weight to all classes regardless of their size. The model’s macro precision of 95.65% means that, on average, when the model predicts a certain class, it is correct about 95.65% of the time. The macro recall of 95.29% shows that the model detects approximately 95.29% of all actual instances in each class, indicating good sensitivity. The macro F1-score of 95.43% balances precision and recall, showing consistent performance across different classes.

These macro scores suggest that the model performs well for all classes, not just the most common ones. This is important because some behaviors, like safe driving, appear much more frequently than others such as “Hair and Makeup” or “Talking to Passenger.” The macro metrics indicate that the model can identify less frequent classes with reasonably high accuracy and sensitivity.

Micro-averaged metrics calculate precision, recall, and F1-score by considering all predictions together, weighing by the number of instances in each class. The micro precision, recall, and F1-score of 95.96% show that the model maintains a high level of accuracy across all predictions in the dataset. The similarity of these micro metrics suggests a good balance between correctly identifying true instances and avoiding false alarms.

The small difference between the macro and micro scores, with macro being slightly lower, indicates that the model performs somewhat better in the more common classes compared to

the less frequent or more difficult ones. This is a common pattern in multi-class classification problems and points to areas where further improvements could focus on enhancing detection of minority classes.

Table 9: Macro and Micro Averaged Metrics and Accuracy

Metric	Value
Accuracy	98,99%
Macro Precision	95,65%
Macro Recall	95,29%
Macro F1-Score	95,43%
Micro Precision	95,96%
Micro Recall	95,96%
Micro F1-Score	95,96%

4.3 Experiment

This experiment was designed to evaluate the practical performance of the YOLO-based driver distraction detection model under real-world conditions, rather than to serve as a formal scientific assessment due to limited resources, such as computational power, special facilities, the setup prioritized feasibility and operational testing.

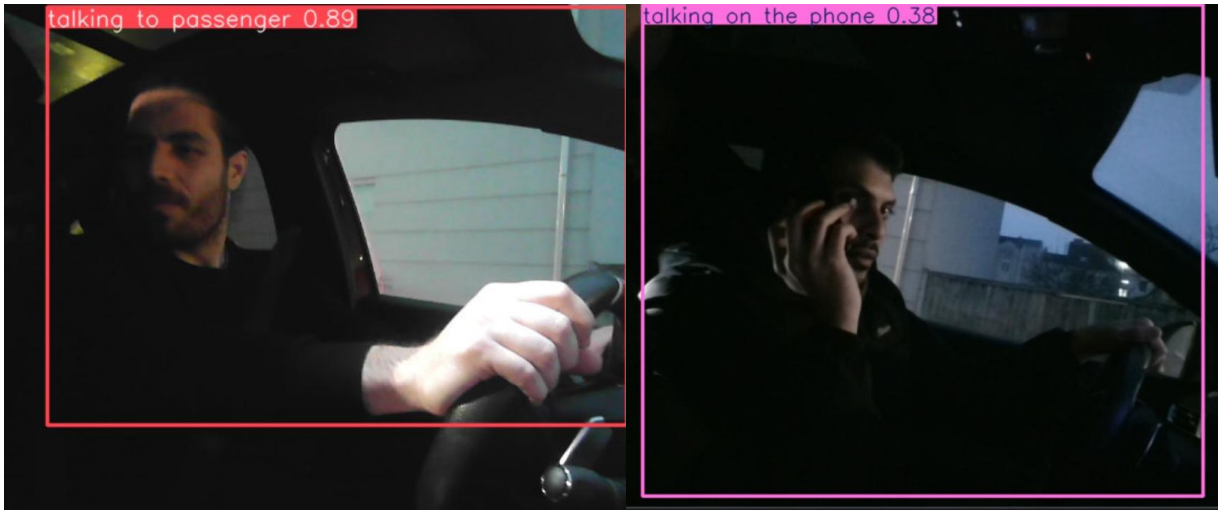


Figure 8: Example frames captured during experiment

A camera installed in a stationary car. For safety reasons, the vehicle remained parked, and the participants acted out distracted driving behaviors instead of driving. The results indicated that the camera angle plays a significant role in the model's performance. Incorrect camera placement seemed to limit the visibility of certain actions and gestures, which could negatively impact detection performance.

The experiment also showed that lighting conditions could affect the model's performance, as it shown in Figure 8. It was observed that the model struggled to maintain accuracy under afternoon lighting, which introduced challenges such as shadows. While these conditions are less difficult than nighttime lighting, they still affect the model's effectiveness. This suggests that the model may need to be improved to handle a variety of lighting situations.

Misclassifications were noted, especially for behaviors that looked visually similar. This finding suggests that relying exclusively on a camera-based YOLO model might have limitations in certain cases. To improve accuracy, incorporating additional sensors, such as infrared sensors, could provide more contextual information and help differentiate between similar classes.

In this experiment, one limiting factor was the modest computational capability of the hardware used, a standard laptop. Such a device is not optimized for high-performance tasks like real-time DL inference and may lack the processing resources typically required to run object detection models such as YOLO efficiently. Additionally, the laptop operated solely on battery power during testing. Many laptops reduce CPU and GPU performance when not connected to a power source. This power-saving behavior may have further constrained the system’s processing capabilities, potentially affecting both inference speed and the consistency of results

To accommodate these limitations, the input resolution was reduced to decrease the computational load. Lower-resolution inputs require less processing power, which enabled the model to function within the constraints of the available hardware. However, this compromise introduced potential trade-offs. With reduced resolution, the model may have more difficulty identifying small or subtle features, for example, hand movements, facial orientation, or small objects like a phone. These details are often important for accurately detecting different forms of driver distraction, and their loss could lead to decreased overall detection accuracy.

5 Conclusion

Driver distraction is a major cause of traffic accidents. It occurs when a driver’s attention shifts away from the task of driving. With advances in technology, machine learning and real-time data can help predict and reduce the risks associated with distracted driving.

This thesis investigates the use of the YOLOv11 DL model in detecting and classifying different driver distraction scenarios, including safe driving, drinking, texting, talking on the phone, applying makeup, reaching behind, operating the radio, and conversing with passengers. The goal was to assess the model’s effectiveness in improving road safety by accurately recognizing these behaviors.

The results demonstrate that the YOLOv11-based model achieves strong and balanced performance across most distraction categories, with high precision, recall, and F1-scores. It performs especially well for behaviors with clear visual features, while performance is slightly lower for more ambiguous actions. The high macro and micro metrics indicate effective detection of both common and less frequent behaviors. Additionally, the consistent decrease in training and validation losses, coupled with an appropriate learning rate schedule, suggests the model generalizes well without overfitting.

Although there is potential for improvement in recognizing less distinctive behaviors, the findings indicate that the model is a robust and reliable tool for driver behavior classification. These results support the potential of machine learning techniques like YOLOv11 to contribute to mitigating distracted driving and improving road safety.

Future Direction:

The YOLOv11 model shows good potential for detecting distracted driving behaviors, but there are still areas where accuracy and reliability can be improved. One area for improvement is the optimization of hyperparameters, which can affect the model's performance. Additionally, some driver behaviors are difficult to distinguish because they share similar visual features and subtle signs. To address this, the system could use multiple types of sensors. By combining visual data from cameras with infrared sensors, eye-tracking devices, the system could gather more accurate information, which would help reduce errors in detection.

Furthermore, a complete safety system could be created by linking the YOLOv11 model to actions that could protect drivers and improve road safety. This system could provide real-time alerts to drivers about possible distractions or even take automatic actions, such as adjusting the vehicle’s speed, activating emergency braking, or sending warnings to nearby vehicles. Additionally, a feedback loop could be included, allowing the model to learn from new data and adjust to emerging patterns of driver distraction.

However, there are challenges to consider, such as the need for proper synchronization of sensors and the merging of data from different sources. Also, privacy and ethical issues related to the use of in-vehicle sensors, like cameras, should be carefully addressed to ensure that data collection meets privacy standards. By combining optimization methods, sensor integration, and safety interventions, future research could significantly enhance distracted driving detection systems, making them more accurate, adaptable, and able to respond effectively to real-world situations.

References

[1] “Road traffic injuries,” Dec. 2023. Accessed: Nov. 28, 2024. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/road-traffic-injuries

[2] International Transport Forum, “Austria: Road Safety Country Profile 2023,” Paris, 2024.

[3] S. G. Klauer, “The impact of driver inattention on near-crash/crash risk: an analysis using the 100-car naturalistic driving study data,” 2006.

[4] P. O. F. VICTORIA, “Inquiry into lowering the probationary driving age in Victoria to seventeen,” 2017.

[5] M. A. Regan, J. D. Lee, and K. Young, *Driver distraction: Theory, effects, and mitigation*. CRC press, 2008.

[6] Canada. T. Canada, Canada. T. Canada. R. Safety, M. V. R. Directorate. S. Research, and D. Branch, *Strategies for reducing driver distraction from in-vehicle telematics devices: a discussion document*. Transport Canada, 2003.

[7] D. L. Strayer, F. A. Drews, and W. A. Johnston, “Cell phone-induced failures of visual attention during simulated driving.,” *J Exp Psychol Appl*, vol. 9, no. 1, p. 23, 2003.

[8] G. M. Fitch *et al.*, “The impact of hand-held and hands-free cell phone use on driving performance and safety-critical event risk,” 2013.

[9] “National Center for Statistics and Analysis (Distracted driving),” 2021.

[10] M. Vegega, B. Jones, and C. Monk, “Understanding the effects of distracted driving and developing strategies to reduce resulting deaths and injuries: a report to congress,” United States. Office of Impaired Driving and Occupant Protection, 2013.

[11] T. A. Ranney, “Driver distraction: A review of the current state-of-knowledge,” 2008.

[12] M. Karthaus, E. Wascher, M. Falkenstein, and S. Getzmann, “The ability of young, middle-aged and older drivers to inhibit visual and auditory distraction in a driving simulator task,” *Transp Res Part F Traffic Psychol Behav*, vol. 68, pp. 272–284, 2020.

[13] Directorate General for Transport, “European Commission, Driver Distraction,” Sep. 2015.

[14] A. Theofilatos, A. Ziakopoulos, E. Papadimitriou, and G. Yannis, “How many crashes are caused by driver interaction with passengers? A meta-analysis approach,” *J Safety Res*, vol. 65, pp. 11–20, 2018.

[15] National Institutes of Health, “Reaching for objects while driving may raise teen crash risk nearly sevenfold,” U.S. Department of Health & Human Services.

[16] V. P. Sisiopiku *et al.*, “Digital advertising billboards and driver distraction,” 2015.

[17] “Driver Distraction: A Review of the Current State-of-Knowledge,” Apr. 2010.

[18] H. V. Koay, J. H. Chuah, C.-O. Chow, and Y.-L. Chang, “Detecting and recognizing driver distraction through various data modality using machine learning: A review, recent advances, simplified framework and open challenges (2014–2021),” *Eng Appl Artif Intell*, vol. 115, p. 105309, 2022, doi: https://doi.org/10.1016/j.engappai.2022.105309.

[19] M. Taherisadr, O. Dehzangi, and H. Parsaei, “Single channel EEG artifact identification using two-dimensional multi-resolution analysis,” *Sensors*, vol. 17, no. 12, p. 2895, 2017.

[20] A. Fernández, R. Usamentiaga, J. L. Carús, and R. Casado, “Driver distraction using visual-based sensors and algorithms,” *Sensors*, vol. 16, no. 11, p. 1805, 2016.

[21] J. D. Lee *et al.*, “Distraction detection and mitigation through driver feedback,” 2013.

[22] W. Zhou, Z. Jia, C. Feng, H. Lu, F. Lyu, and L. Li, “Towards driver distraction detection: a privacy-preserving federated learning approach,” *Peer Peer Netw Appl*, vol. 17, no. 2, pp. 896–910, 2024.

[23] M.-Y. Fu and Y.-S. Huang, “A survey of traffic sign recognition,” in *2010 International Conference on Wavelet Analysis and Pattern Recognition*, 2010, pp. 119–124. doi: 10.1109/ICWAPR.2010.5576425.

[24] J. W. Lee, “A Machine Vision System for Lane-Departure Detection,” *Computer Vision and Image Understanding*, vol. 86, no. 1, pp. 52–78, 2002, doi: https://doi.org/10.1006/cviu.2002.0958.

[25] P. Dollar, C. Wojek, B. Schiele, and P. Perona, “Pedestrian Detection: An Evaluation of the State of the Art,” *IEEE Trans Pattern Anal Mach Intell*, vol. 34, no. 4, pp. 743–761, 2012, doi: 10.1109/TPAMI.2011.155.

[26] L.-C. Fu and C.-Y. Liu, “Computer vision based object detection and recognition for vehicle driving,” in *Proceedings 2001 ICRA. IEEE International Conference on Robotics and Automation (Cat. No.01CH37164)*, 2001, pp. 2634–2641 vol.3. doi: 10.1109/ROBOT.2001.933020.

[27] H. Song, “The Application of Computer Vision in Responding to the Emergencies of Autonomous Driving,” in *2020 International Conference on Computer Vision, Image and Deep Learning (CVIDL)*, 2020, pp. 1–5. doi: 10.1109/CVIDL51233.2020.00008.

[28] F. Qu, N. Dang, B. Furht, and M. Nojoumian, “Comprehensive study of driver behavior monitoring systems using computer vision and machine learning techniques,” *J Big Data*, vol. 11, no. 1, p. 32, 2024.

[29] M. F. Majeed and A. A. Fahad, “Traffic control system techniques: A review,” *International Journal of Nonlinear Analysis and Applications*, vol. 13, no. 2, pp. 829–835, 2022.

[30] V. Mandal, A. R. Mussah, P. Jin, and Y. Adu-Gyamfi, “Artificial intelligence-enabled traffic monitoring system,” *Sustainability*, vol. 12, no. 21, p. 9177, 2020.

[31] D. Chowdhury, S. Mandal, D. Das, S. Banerjee, S. Shome, and D. Choudhary, “An Adaptive Technique for Computer Vision Based Vehicles License Plate Detection System,” in *2019 International Conference on Opto-Electronics and Applied Optics (Optronix)*, 2019, pp. 1–6. doi: 10.1109/OPTRONIX.2019.8862406.

[32] A. Géron, *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow*. “ O’Reilly Media, Inc.,” 2022.

[33] S. Ray, “A quick review of machine learning algorithms,” in *2019 International conference on machine learning, big data, cloud and parallel computing (COMITCon)*, IEEE, 2019, pp. 35–39.

[34] G. Biau and E. Scornet, “A random forest guided tour,” *Test*, vol. 25, no. 2, pp. 197–227, 2016.

[35] Bruce Blaus, “Neuron.”

[36] T. E. Potok *et al.*, “A study of complex deep learning networks on high-performance, neuromorphic, and quantum computers,” *ACM Journal on Emerging Technologies in Computing Systems (JETC)*, vol. 14, no. 2, pp. 1–21, 2018.

[37] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, no. 7553, pp. 436–444, 2015, doi: 10.1038/nature14539.

[38] L. Alzubaidi *et al.*, “Review of deep learning: concepts, CNN architectures, challenges, applications, future directions,” *J Big Data*, vol. 8, pp. 1–74, 2021.

[39] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.

[40] B. Van Essen, H. Kim, R. Pearce, K. Boakye, and B. Chen, “LBANN: Livermore big artificial neural network HPC toolkit,” in *Proceedings of the workshop on machine learning in high-performance computing environments*, 2015, pp. 1–6.

[41] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[42] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “Imagenet classification with deep convolutional neural networks,” *Adv Neural Inf Process Syst*, vol. 25, 2012.

[43] K. Simonyan, “Very deep convolutional networks for large-scale image recognition,” *arXiv preprint arXiv:1409.1556*, 2014.

[44] R. Girshick, J. Donahue, T. Darrell, and J. Malik, “Rich feature hierarchies for accurate object detection and semantic segmentation,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2014, pp. 580–587.

[45] R. Girshick, “Fast R-CNN,” in *2015 IEEE International Conference on Computer Vision (ICCV)*, 2015, pp. 1440–1448. doi: 10.1109/ICCV.2015.169.

[46] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards real-time object detection with region proposal networks,” *IEEE Trans Pattern Anal Mach Intell*, vol. 39, no. 6, pp. 1137–1149, 2016.

[47] J. Redmon, “You only look once: Unified, real-time object detection,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016.

[48] Dr. Weidong Kuang, “Fundamentals of deep learning: a step-by-step guide,” ch. Chapter 10.

[49] N. Jegham, C. Y. Koh, M. Abdelatti, and A. Hendawi, “Evaluating the Evolution of YOLO (You Only Look Once) Models: A Comprehensive Benchmark Study of YOLO11 and Its Predecessors,” *arXiv preprint arXiv:2411.00201*, 2024.

[50] H. M. Eraqi, Y. Abouelnaga, M. H. Saad, and M. N. Moustafa, “Driver distraction identification with an ensemble of convolutional neural networks,” *J Adv Transp*, vol. 2019, no. 1, p. 4125865, 2019.

[51] S. Farm, “State farm distracted driver detection,” Technical Report. 2016. Available online: https://www. kaggle. com/c/state …, 2016.

[52] Y. Abouelnaga, H. M. Eraqi, and M. N. Moustafa, “Real-time distracted driver posture classification,” *arXiv preprint arXiv:1706.09498*, 2017.

[53] S. F. Ali and M. T. Hassan, “Feature based techniques for a driver’s distraction detection using supervised learning algorithms based on fixed monocular video camera,” *KSII Transactions on Internet and Information Systems (TIIS)*, vol. 12, no. 8, pp. 3820–3841, 2018.

[54] I.-H. Choi, S. K. Hong, and Y.-G. Kim, “Real-time categorization of driver’s gaze zone using the deep learning techniques,” in *2016 International conference on big data and smart computing (BigComp)*, IEEE, 2016, pp. 143–148.

[55] R. C.-H. Chang, C.-Y. Wang, and C.-H. Shen, “Modified YOLOv3-Tiny Using Dilated Convolution for Driver Distraction Detection,” in *2020 IEEE International Conference on Consumer Electronics-Taiwan (ICCE-Taiwan)*, IEEE, 2020, pp. 1–2.

[56] Y.-S. Poon *et al.*, “Driver distracted behavior detection technology with YOLO-based deep learning networks,” in *2021 IEEE International Symposium on Product Compliance Engineering-Asia (ISPCE-ASIA)*, IEEE, 2021, pp. 1–5.

[57] F. Sajid, A. R. Javed, A. Basharat, N. Kryvinska, A. Afzal, and M. Rizwan, “An efficient deep learning framework for distracted driver detection,” *IEEE Access*, vol. 9, pp. 169270–169280, 2021.

[58] K. Ataş and R. A. Vural, “Detection of driver distraction using yolov5 network,” in *2021 2nd Global Conference for Advancement in Technology (GCAT)*, IEEE, 2021, pp. 1–6.

[59] Y. Wei *et al.*, “Distracted driver behavior detection based-on an improved yolox framework,” in *2022 27th International Conference on Automation and Computing (ICAC)*, IEEE, 2022, pp. 1–6.

[60] Z. Wang, K. Yao, and F. Guo, “Driver attention detection based on improved yolov5,” *Applied Sciences*, vol. 13, no. 11, p. 6645, 2023.

[61] Y. Zheng, S. Chen, J. Wu, K. Chen, T. Wang, and T. Peng, “Real-Time Driver Fatigue Detection Method Based on Comprehensive Facial Features,” in *International Conference on Algorithms and Architectures for Parallel Processing*, Springer, 2023, pp. 484–501.

[62] E. Murali, C. G. Vignesh, and G. P. Varma, “Driver Drowsiness Detection Using YOLO,” in *2024 10th International Conference on Advanced Computing and Communication Systems (ICACCS)*, IEEE, 2024, pp. 1608–1612.

[63] B. Ma *et al.*, “Distracted Driving Behavior and Driver’s Emotion Detection Based on Improved YOLOv8 with Attention Mechanism,” *IEEE Access*, 2024.

[64] I. Markoulidakis, G. Kopsiaftis, I. Rallis, and I. Georgoulas, “Multi-class confusion matrix reduction method and its application on net promoter score classification problem,” in *Proceedings of the 14th PErvasive Technologies Related to Assistive Environments Conference*, 2021, pp. 412–419.

List of figures

Figure 1: A biological neuron [32], [35]. 21
Figure 2: Object Detection in Action: A YOLOv2 Example[48] 29
Figure 3: the generic architecture of YOLO[48] 30
Figure 4: The backbone extracts one feature vector for each 7 × 7 grid cell[48]. 30
Figure 5: YOLO v1 Prediction Tensor. (Left) Tensor Shape (Right) Example of x, y, w, h 30
Figure 6: Intersection over Union (IOU) 31
Figure 7: Training and validation losses, metrics, and learning rate across epochs 52
Figure 8: Example frames captured during experiment 56

List of tables

Table 1: Levels of vehicle automation[18]. 8
Table 2: Summary of Physiological Sensors 9
Table 3: Summary of external sensors 12
Table 4: Key components of neural network architecture 25
Table 5: The structure of a multi-class classification confusion matrix 47
Table 6: Training and Validation Losses Across Epochs 52
Table 7: Confusion matrix 53
Table 8: Class-wise classification metrics 53
Table 9: Macro and Micro Averaged Metrics and Accuracy 56

Formula directory

Localization loss (1)

$$\lambda_{coord}\sum_{i=1}^{S^2}\sum_{j=1}^{B}\theta_{ij}^{obj}[(x_i-\hat{x}_i)^2+(y_i-\hat{y}_i)^2]$$
$$+\lambda_{coord}\sum_{i=1}^{S^2}\sum_{j=1}^{B}\theta_{ij}^{obj}\left[\left(\sqrt{w_i}-\sqrt{w}\right)^2+\left(\sqrt{h_i}-\sqrt{\hat{h}_i}\right)^2\right]$$

Confidence loss (For boxes responsible for detecting an object) (2)

$$\sum_{i=1}^{S^2}\sum_{j=1}^{B}\theta_{ij}^{obj}(c_i-\hat{c}_i)^2$$

Confidence loss (For boxes not responsible for detecting an object) (3)

$$\lambda_{noord}\sum_{i=1}^{S^2}\sum_{j=1}^{B}\theta_{ij}^{nobj}(c_i-\hat{c}_i)^2$$

Classification loss (4)

$$\sum_{i=1}^{S^2}\theta_i^{obj}\sum_{c\epsilon classes}\left(p_i(c)-\hat{p}_i(c)\right)^2$$

Total loss (5)

$$Total\ Loss\ =\ Localization\ Loss+Confidence\ Loss+Classification\ Loss$$

Overall Accuracy (6)

$$Accuracy=\frac{\sum_{i=1}^{N}\mathrm{TP}\ (\mathrm{C_i})}{\sum_{i=1}^{N}\sum_{j=1}^{N}C_{i,j}}$$

Class-Specific Recall (True Positive Rate - TPR) (7)

$$Recall\ of\ class\ C_i=TPR(C_i)\ =\frac{TP\ (C_i)}{\mathrm{TP}\ (C_i)+FN\ (C_i)}$$

Class-Specific Precision (Positive Predictive Value - PPV) (8)

$$Precision\ of\ class\ C_i=PPV\ (C_i)\ =\frac{TP\ (C_i)}{TP\ (C_i)+FP\ (C_i)}$$

Class-Specific F1-Score (9)

$$F1-Score\ of\ class\ C_i=F_1(C_i)\ =\frac{2*TPR(C_i)*PPV\ (C_i)}{TPR(C_i)+PPV\ (C_i)}$$

Macro-Averaged Recall (10)

$$Recall\ (macro\ average)=TPR\ (macro)=\frac{1}{N}\sum_{i=1}^{N}TPR(C_i)$$

Macro-Averaged Precision (11)

$$Precision\ (macro\ average) = PPV\ (macro) = \frac{1}{N}\sum_{i=1}^{N} PPV(C_i)$$

Macro-Averaged F1-Score (12)

$$F1 - Score\ (\text{macro average}) = F_1(macro) = \frac{2 * TPR(macro) * PPV\ (macro)}{TPR(macro) + PPV\ (macro)}$$

Micro-Averaged Recall (13)

$$Recall\ (micro\ average) = TPR\ (micro) = \frac{\sum_{i=1}^{N} TP\ (C_i)}{\sum_{i=1}^{N}[TP\ (C_i) + FN\ (C_i)]}$$

Micro-Averaged Precision (14)

$$Precision\ (micro\ average) = PPV\ (micro) = \frac{\sum_{i=1}^{N} TP\ (C_i)}{\sum_{i=1}^{N}[TP\ (C_i) + FP\ (C_i)]}$$

F1-Score (Micro Average) (15)

$$F1 - Score\ (micro\ average) = F_1(micro) = \frac{2 * TPR(micro) * PPV\ (micro)}{TPR\ (micro) + PPV\ (micro)}$$

List of abbreviations

deep learning (DL)
computer vision (CV)
you only look once (YOLO)
machine learning (ML)
electroencephalogram (EEG)
electrocardiogram (ECG)
electrooculography (EOG)
electro-dermal activity (EDA
electromyography (EMG)
convolutional neural network (CNN)
Intersection over Union (IOU)
spatial pyramid pooling fast (SPFF)
mean average precision (mAP)
true positive (TP)
true negatives (TN)
false positives (FP)
false negatives (FN)